

Technical Reverse Engineering and Architectural Masterclass: Archonet Planwise Site Development System

Part 1: Product and Business Domain

The Real Estate Feasibility Problem

In conventional real estate development, evaluating the financial and physical viability of an acquisition parcel requires between four and six weeks of manual effort. Developers, architects, and civil engineering consultants traditionally operate across disconnected workflows: municipal planning regulations, such as the Development Control and Promotion Regulations for Greater Mumbai (DCPR 2034) or the Unified Development Control and Promotion Regulations for Maharashtra State (UDCPR), are parsed manually from dense statutory legal texts. Physical property boundaries are sketched from scanned revenue surveys or isolated CAD files, while financial pro-formas are calculated in static spreadsheets disconnected from physical spatial boundaries.

This latency creates significant commercial friction. Land acquisition in dense urban markets is highly capital-intensive and time-sensitive. If an acquisition team cannot determine within days whether an irregular parcel can support an efficient Floor Space Index (FSI) after deducting mandatory setbacks and reservations, they either forfeit bidding opportunities or commit capital to unviable land. Archonet Planwise was conceived within Archonet's software ecosystem—alongside LandWise for regulatory estimation and FinWise for financial forecasting—to digitize, automate, and accelerate site-level feasibility. By providing a browser interface that links interactive geospatial canvas manipulation with municipal parameters and generative layout algorithms, Planwise compresses early-stage feasibility analysis from weeks down to minutes.

Target Users and Commercial Personas

The system targets three distinct professional profiles across the real estate lifecycle:

1. **Real Estate Developers:** Executive decision-makers and land acquisition leads who require immediate go/no-go screening metrics, gross development potential, and spatial risk assessments before executing formal letters of intent or joint-venture agreements.
2. **Architects and Master Planners:** Design professionals who need to replace manual massing drafts with automated boundary definitions, buildable envelope calculations, setback offsets, and tree preservation buffers.
3. **Real Estate Advisory Firms and Land Aggregators:** Transaction intermediaries responsible for presenting data-backed land feasibility prospectuses to institutional investors and funds.

Project Definition and Feasibility Context

Within Planwise, a "Project" represents an encapsulated spatial, legal, and regulatory record tied to a specific geographic parcel. It serves as the single source of truth for all downstream layout calculations. A project aggregates:

- Metadata, including the administrative title, narrative description, and creation timestamps.
- Land Ownership classification, which maps to statutory categories such as Private Land (Freehold/Leasehold), Collector Land (Class II), Collector Land (Leasehold), Estate Land (Leasehold), or MHADA Land.
- Spatial Geometries, including the outer parcel boundary polygon, interior non-developable area polygons, and point coordinates of preserved trees.
- Contextual Constraints, including municipal access road widths, perimeter setbacks, height limitations, and parking allowances.

Feasibility analysis in this context refers to determining the net buildable footprint and spatial efficiency of a plot. In dense urban environments, two parcels possessing identical gross area and permissible FSI often exhibit drastically different financial viability. Mandatory perimeter setbacks, road-widening reservations, internal turning radii for fire tenders, and non-developable zones (such as water bodies, steep slopes, high-tension lines, or protected flora) reduce the gross site area to a net developable envelope. If the resulting footprint is excessively narrow or fragmented, the core-to-floorplate ratio balloons, reducing net saleable efficiency and destroying equity returns. Planwise provides the graphical and mathematical engine to determine this developable envelope accurately.

The Necessity of Land Parcel Digitization

Manual land surveys and CAD drawings are inert; they cannot natively trigger reactive calculation pipelines in web environments. Digitizing land parcels into vector coordinates using the RFC 7946 GeoJSON standard unlocks real-time client-side analysis:

- Automated Area Computation: Spherical polygon area calculations occur instantly in the browser as the user draws or modifies vertices.
- Computational Geometry Operations: Spatial clipping, buffer calculations for setbacks, and boolean subtractions for non-developable zones can be executed algorithmically.
- Serialization and Interoperability: Coordinate payloads are passed via lightweight JSON payloads to cloud generation microservices, eliminating heavy desktop drafting software.

Map Roles, Environmental Context, and Constraints

The map functions as an active spatial input canvas rather than a static background image. It provides real-world contextual data, including satellite imagery, street networks, and civic infrastructure, against which developers map physical realities:

- Regulatory Constraints: Municipal building codes impose strict constraints based on adjacent road widths and parcel classifications. For example, under Mumbai's DCPR 2034, road width dictates maximum permissible tower height, fire tender access margins, and premium FSI loading. The frontend captures these parameters via structured forms (GetRegulatoryForm) and projects their implications onto the spatial model.
- Business Constraints: Captured via BusinessConstraints overlays, these encompass developer-specified targets, such as commercial-to-residential unit ratios, target carpet areas, parking typology preferences (podiums versus multi-level basements), and project phasing strategies.

- **Trees and Non-Developable Areas:** Environmental preservation mandates that mature trees cannot be arbitrarily felled; they impose exclusion zones that alter building footings. Similarly, existing easements, public reservations, or utility corridors represent non-developable polygons that must be subtracted from the total plot boundary.

Major Workflows Supported by the Planwise UI

The Planwise interface is divided into three consecutive planning workflows selected via the ProjectSelection component:

1. **Site Feasibility:** The user searches for a site location (restricted to India via Mapbox Geocoder), delineates the outer boundary using polygon drawing tools, marks exclusion zones, places tree markers, and enters regulatory constraints. Real-time metrics compute the gross and net area.
2. **Residential Unit Layout:** Downstream generative layout tools utilize the net buildable envelope to synthesize structural column grids, corridors, and modular residential units (1BHK, 2BHK, 3BHK) while respecting daylight and egress norms.
3. **Building Massing:** The 2D net layout is extruded into 3D structural volumes across vertical tiers, incorporating podiums, transfer slabs, and tower envelopes to evaluate FSI consumption against municipal height caps.

End-to-End Real-World Walkthrough

To contextualize the system's operational lifecycle:

1. **Authentication:** An acquisition lead navigates to /login, inputs credentials, and receives a signed JWT, which the client persists in browser storage.
2. **Dashboard Navigation:** The system redirects the user to /home, where the Projects component queries the API and renders a paginated grid of existing development parcels.
3. **Project Initialization:** The user clicks "Create New Project," opening the NewProject modal. They enter the title ("Bandra Kurla Residential Feasibility"), an operational summary, and select "Private Land (Freehold/Leasehold)" from the land ownership dropdown. Upon submission, the API responds with HTTP 201 Created and persists the project record.
4. **Workflow Selection:** The user chooses the "Site Feasibility" workflow card within ProjectSelection, which routes them to the interactive MapPage with project identity preserved in navigation state.
5. **Spatial Boundary Definition:** On MapPage, the user searches for "Bandra East, Mumbai" using the integrated Mapbox Geocoder. The camera navigates to the coordinates. The user selects the Polygon tool from the Toolbar, drawing the parcel perimeter across four vertices. Mapbox Draw fires events that calculate parcel area and render it within a fixed heads-up UI panel.
6. **Feature Placement:** The boundary state transitions the toolbar to enable environmental tools. The user toggles the Tree tool, clicking within the parcel to instantiate two tree markers representing preserved banyan trees. An interior utility easement is outlined using the Non-Developable Area polygon tool.
7. **Auto-Save and Constraints:** The internal debouncing mechanism captures these spatial updates and issues a background PUT/POST payload updating the GeoJSON data. The user toggles the regulatory form overlay, enters an adjacent road width of 18.0 meters, transitions to the business constraints form to specify an 80:20 residential-to-retail mix,

and reviews the consolidated InputSummary before confirming and navigating to /site-constraints for generative layout generation.

Part 2: Complete System Architecture

Architectural Topology and Layer Decomposition

The Planwise platform is structured as a decoupled Single Page Application (SPA) communicating over HTTPS with a RESTful backend API service. The client application coordinates graphical spatial manipulation in a WebGL canvas with traditional document-object-model form states.

Architectural Layer	Technical Implementation	Responsibilities	Data Ingress	Data Egress	Failure Modes	Debugging Approaches	Source Status
1. Client Presentation	Modern Web Browser (V8/Blink, Gecko, WebKit)	Renders HTML5 DOM and paints WebGL context onto canvas; captures pointer and keyboard events.	User peripheral interactions (mouse clicks, coordinate inputs, form entry).	Low-level DOM events (click, change, pointerdown, keydown).	WebGL context crashes, hardware acceleration disabled, CSS layout shifts.	Inspect browser console, toggle Chrome WebGL flags (chrome://gpu), test responsive breakpoints.	Explicitly Documented
2. React Component Hierarchy	React.js SPA (Functional Components)	Houses tree structure: Login, Projects, NewProject, MapPage, Toolbar, overlays.	Raw DOM events, parent component props, React Context.	Virtual DOM patches, local hook state changes, HTTP request triggers.	Component re-render cascades, unmounted state updates, missing prop drilling.	React DevTools Profiler, React Component Tree inspection, breakpoint debugging.	Explicitly Documented
3. Client State & Ref Engine	React Hooks (useState, useEffect, useRef)	Maintains synchronous component state, form inputs, modal flags, and holds mutable Mapbox instances	User input values, API response payloads, Mapbox Draw event data.	Cleaned state objects, debounced update signals, prop payloads to children.	Stale state closures in async calls, uncontrolled re-renders resetting map canvas, memory leaks.	React DevTools Hooks inspector, console logging within dependency arrays, why-did-you-render.	Explicitly Documented

Architectural Layer	Technical Implementation	Responsibilities	Data Ingress	Data Egress	Failure Modes	Debugging Approaches	Source Status
		without triggering re-renders.					
4. Geospatial Canvas Engine	Mapbox GL JS & Mapbox Draw Plugin	Hardware-accelerated WebGL cartographic rendering, vector tile parsing, dynamic GeoJSON layer visualization, polygon drafting.	Screen coordinate clicks, vector tile sets, raw GeoJSON Feature objects.	Normalized WGS84 GeoJSON features, bounding box dimensions, camera zoom/pitch events.	GeoJSON self-intersections, Mapbox access token expiration, canvas container 0px height collapse.	Mapbox Debug mode (map.showTileBoundaries = true), inspecting draw.getAll() directly in console.	Explicitly Documented
5. HTTP Transport Layer	Axios & Window Fetch Client	Asynchronous Promise-based HTTP networking; serializes payloads, orchestrates debounced requests, manages retry cycles.	JavaScript objects, spatial JSON strings, target URL endpoints.	Serialized HTTP requests (application/json), resolved response Promises.	Network timeouts, CORS preflight rejections, malformed JSON stringification.	Chrome Network tab filtering (XHR/Fetch), Axios request/response interceptor logging.	Explicitly Documented
6. Security & Session Gateway	JSON Web Tokens (Bearer Token Pattern)	Intercepts HTTP requests to inject Authorization: Bearer <token>; guards React Router paths based on localStorage	Raw login credentials, stored JWT strings from browser storage.	Authorized HTTP request headers, route transition authorizations.	Expired tokens failing silently, CSRF vulnerabilities, token desynchronization on multi-tab use.	Application Tab -> Local Storage inspection, decoding token at jwt.io to check expiry (exp).	Explicitly Documented

Architectural Layer	Technical Implementation	Responsibilities	Data Ingress	Data Egress	Failure Modes	Debugging Approaches	Source Status
		e tokens.					
7. Application API Layer	RESTful HTTP Service (Target: localhost:5001)	Validates incoming payloads, unpacks GeoJSON stringified blobs, enforces business logic, manages route controllers.	HTTP requests containing project parameters, GeoJSON structures, user IDs.	Structured JSON responses (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized).	Controller exceptions, unhandled JSON parsing errors, database connection dropouts.	Postman API suite verification, backend process logs, integration test suites.	Reasonable Inference / Documented Endpoints
8. Persistence Layer	Relational / Document Store (e.g., PostgreSQL with PostGIS or MongoDB)	Long-term durable storage of user credentials, project metadata, constraints, and spatial geometries.	Normalized SQL insert statements / BSON documents.	Persisted entity records, relational joins, spatial query matches.	Coordinate truncation, invalid geometry data types, deadlocks, connection pool exhaustion.	Database CLI queries, spatial indexing validation (ST_IsValid), explain-plan query analysis.	Unknown Backend Details

Inter-Layer Communication and Protocol Mechanics

Data traverses the architecture through strictly defined protocols:

1. **Interaction to State:** The user clicks on the Mapbox canvas. Low-level WebGL mouse event listeners translate screen pixels into spherical latitude and longitude (EPSG:4326). Mapbox Draw packages these into a GeoJSON Feature.
2. **State to Ref:** Because redrawing a React component unmounts and remounts child DOM elements—which would destroy the active WebGL canvas context—the Mapbox instance is stored in a React useRef. State changes regarding tool modes or metrics updates are decoupled from the canvas lifecycle.
3. **State to Transport:** Changes to the boundary or markers trigger a debounced dispatch function. This batches rapid vertex adjustments into a single consolidated GeoJSON object, serializing arrays into strings where specified by the documented API schema ("geojson": "{...}", "trees": "[...]"), and passes them to an Axios instance.
4. **Transport to Network:** Axios injects the Authorization header retrieved from localStorage (Bearer eyJhbGciOi...) and executes an asynchronous HTTP POST or PUT across TCP/IP to the target port 5001.
5. **API to Database:** The backend parses the Authorization header, validates the signature using an internal secret key, validates the GeoJSON geometry against structural

schemas, and issues persistence transactions to the database.

Documented Implementation Details Versus Unknowns

To ensure technical integrity, system boundaries must be clearly delineated:

- Documented in the Internship Report: The frontend stack (React, React Router, Mapbox GL JS, Mapbox Draw, Mapbox Geocoder, Axios, React Toastify, CSS Modules/SCSS); the component names and their behavioral requirements; the specific local API endpoint (<http://localhost:5001/api/projects>); the exact project creation JSON payload format; and the usage of localStorage for JWT persistence.
- Reasonably Inferred: The API architecture is a Node.js/Express or Python/Flask REST service running locally on port 5001, given the JSON response structures, route naming conventions (</api/projects>), and standard full-stack patterns used in modern web engineering.
- Unknown in the Source: The exact database technology (whether a PostgreSQL database with PostGIS spatial extensions, a document-based MongoDB instance, or an open-source BaaS like Supabase as mentioned in the report's reference section); the backend programming language and framework; the private key signing algorithms for JWTs; and the downstream massing/layout generative calculation engine algorithms.

Part 3: Component-by-Component Reverse Engineering

Component Hierarchy and Architectural Relationship

The application structure is organized as a declarative component tree. The root application router manages top-level navigation, using protected route wrappers to guard authenticated paths. The primary dashboard hosts the project catalog and creation modals, routing selected projects into the intermediate workflow selector. The core spatial workspace encapsulates the dynamic toolbar, WebGL canvas, HUD metrics readouts, and modal form overlays for regulatory constraints, business constraints, and consolidated pre-flight review.

1. Login Component

- Purpose: Serves as the authentication gateway for the platform, ensuring unauthenticated requests cannot access protected project resources.
- User Interaction: Inputting email and password strings, clicking password visibility toggle icon, clicking "Submit" button.
- State Variables: email (string), password (string), showPassword (boolean), isLoading (boolean), errors (object mapping field names to validation strings).
- Props Supported: None (top-level route-level component).
- Events Handled: onChange on inputs, onClick on visibility toggle, onSubmit on login form.
- API Calls: POST </api/auth/login> (or equivalent auth endpoint) passing { email, password }.
- Data Received: JSON payload containing { token: string, user: { id, email, name } }.
- Data Sent: Credential object { email, password }.
- UI States: Default pristine, editing, invalid inputs with inline alerts, loading spinner on

submit button, global error toast.

- Loading States: When `isLoading` is true, the submit button is disabled and renders an inline spinning indicator.
- Error States: Field-level validation flags (e.g., "Invalid email format"), HTTP 401 Unauthorized messages ("Invalid credentials").
- Success States: HTTP 200/201 response triggers JWT storage and automatic redirect to `/home`.
- Dependencies: React, React Router (`useNavigate`), React Toastify, Axios.
- Relationship with Other Components: Directly controls entry to Projects and all downstream protected paths.
- Likely Lifecycle: Mounts -> checks if valid token already exists in `localStorage` (auto-redirect to `/home` if true) -> idle user input -> submit async cycle -> unmount.
- Potential Bugs: Storing JWT in `localStorage` without validating token expiration allows a user to access the shell UI briefly before API calls return 401.
- Modern Reconstruction Strategy: Implement using React Hook Form coupled with Zod schema validation and an authentication context provider to eliminate prop drilling.

2. Projects Component

- Purpose: Main application dashboard displaying an interactive, paginated catalog of all user-owned development projects.
- User Interaction: Typing into search filter, clicking "Create New Project" card, clicking "Load More" pagination button, clicking project cards to open, clicking action icons (share, download PDF, delete).
- State Variables: `projectsList` (array of project objects), `searchQuery` (string), `visibleCount` (number, initialized to 7), `isLoading` (boolean), `isDeleting` (boolean/string id).
- Props Supported: None (route-level component rendered at `/home`).
- Events Handled: Search `onChange`, pagination `onClick`, delete action `onClick`.
- API Calls: GET `/api/projects` to retrieve parcel collections; DELETE `/api/projects/:id` to purge a project.
- Data Received: Array of project objects containing metadata, ownership type, image URLs, and spatial previews.
- Data Sent: Target project ID string in DELETE requests.
- UI States: Initial skeleton loading, populated grid, filtered empty search state, paginated extension state.
- Loading States: Initial page loading skeleton; spinner on "Load More" button during asynchronous batch fetches.
- Error States: Network failure toasts if fetching projects fails; inline modal confirmation if deletion rejects.
- Success States: Dynamic UI pruning upon successful project deletion; incremental rendering of 4 additional project tiles when "Load More" is clicked.
- Dependencies: React, React Router, Axios, React Toastify, SCSS Modules.
- Relationship with Other Components: Parent dashboard embedding the "Create New Project" action which triggers `NewProject`; passes project context to `ProjectSelection` upon tile click.
- Likely Lifecycle: Mounts -> executes `useEffect` to fetch project array -> populates state -> manages client-side slicing based on `visibleCount` -> updates upon mutations.
- Potential Bugs: Client-side slicing without true server-side pagination leads to heavy

payload downloads if a developer has hundreds of parcels.

- Modern Reconstruction Strategy: Use TanStack Query (React Query) for caching, optimistic updates, and server-side cursor-based infinite scrolling.

3. NewProject Component

- Purpose: Modal form overlay capturing the primary administrative and legal parameters required to initialize a development parcel.
- User Interaction: Entering text in project title and description inputs, selecting ownership category from styled dropdown, clicking modal close or submit buttons.
- State Variables: projectTitle (string), projectDescription (string), landOwnership (string), errors (object), isSubmitting (boolean), successMessage (string).
- Props Supported: isOpen (boolean), onClose (function), onProjectCreated (callback function).
- Events Handled: Form onSubmit, input onChange, dropdown selection onChange.
- API Calls: POST `http://localhost:5001/api/projects`.
- Data Received: HTTP 201 Created confirmation and persisted project entity containing generated `_id/id`.
- Data Sent: `{ project_title, about, land_ownership, geojson: "{}", nondevelopableareas: "[]", trees: "[]" }`.
- UI States: Modal hidden, modal active/open, validation error highlighting, submission disabled.
- Loading States: Submit button disabled with spinner, input fields locked (readOnly or disabled) during API transit.
- Error States: Inline red borders and error text beneath inputs ("Project title is required", "Please select a valid land ownership type").
- Success States: Success message rendered in a green callout box; triggers parent project list refresh and closes modal.
- Dependencies: React, Axios, React Toastify.
- Relationship with Other Components: Invoked by Projects component; upon successful creation, transitions user context forward.
- Likely Lifecycle: Instantiated inside Projects DOM -> opens via state flag -> handles inputs -> validates on submit -> fires API -> resets form -> closes.
- Potential Bugs: Failure to reset state variables upon closing the modal causes stale data to appear if the user reopens the modal without refreshing.
- Modern Reconstruction Strategy: Encapsulate within an accessible dialog primitive (`@radix-ui/react-dialog`) with complete keyboard focus trapping and ARIA attributes.

4. ProjectSelection Component

- Purpose: Intermediate routing and workflow dispatch interface where users select their planning objective for a specific parcel.
- User Interaction: Selecting one of three workflow cards: "Site Feasibility", "Residential Unit Layout", or "Building Massing".
- State Variables: selectedWorkflow (string), projectContext (object holding active project metadata).
- Props Supported: Implicit props via React Router `location.state`.
- Events Handled: Card onClick, navigation triggers.

- API Calls: Optional verification GET /api/projects/:id if location.state is empty on browser reload.
- Data Received: Contextual project parameters (ID, title, boundaries).
- Data Sent: Selected workflow parameters passed forward to routing state.
- UI States: Interactive 3-card layout with active hover and selection states.
- Loading States: Brief transition skeleton if verifying project context from remote storage.
- Error States: Fallback warning dialog ("Project context missing. Please select a project from the dashboard") with a redirect button to /home.
- Success States: Direct programmatic navigation to /map or downstream generative workspaces.
- Dependencies: React, React Router (useLocation, useNavigate).
- Relationship with Other Components: Sits between Projects and MapPage, establishing workflow intent.
- Likely Lifecycle: Mounts -> parses location.state -> validates presence of project_id -> renders options -> routes user on click.
- Potential Bugs: User refreshes browser at /project-selection, wiping volatile React Router location.state, causing an application crash if unhandled.
- Modern Reconstruction Strategy: Use URL dynamic parameters (/projects/:projectId/selection) and synchronize project context via an active project global store or React Query cache.

5. MapPage Component

- Purpose: The primary spatial workspace of Planwise, coordinating the Mapbox GL JS canvas, drawing tools, geocoding search, and constraint overlays.
- User Interaction: Panning, zooming, rotating, 3D tilting, placing polygon vertices, placing tree markers, searching locations, toggling tool modals.
- State Variables: activeTool (string), areaMetrics (object containing gross, net, and deduction values), isAutoSaving (boolean), overlaysOpen (object flags for regulatory, business, and summary modals), mapLoaded (boolean).
- Props Supported: None (route-level component at /map).
- Events Handled: Mapbox event bindings (load, draw.create, draw.update, draw.delete, click, rotate, pitch).
- API Calls: PUT /api/projects/:id (debounced auto-save of GeoJSON, trees, deductions).
- Data Received: Persisted project GeoJSON structures, existing marker coordinates, saved constraints.
- Data Sent: Consolidate GeoJSON FeatureCollections, serialized spatial strings, updated constraint values.
- UI States: Fullscreen canvas with floating dynamic toolbar, heads-up metrics panel, active drawing indicators, modal overlays.
- Loading States: Map initialization spinner; discreet pulsing indicator during background debounced auto-save cycles.
- Error States: Toast alerts on WebGL initialization failures, invalid boundary geometry warnings (e.g., self-intersections), auto-save failure notifications.
- Success States: Real-time updates of calculated area metrics in the HUD panel; toast confirming successful auto-save.
- Dependencies: Mapbox GL JS, Mapbox Draw, Mapbox Geocoder, Turf.js (inferred for measurements), Axios, React Toastify.

- Relationship with Other Components: Hosts Toolbar, GetRegulatoryForm, BusinessConstraints, GetBusinessConstraints, and InputSummary.
- Likely Lifecycle: Mounts -> initializes Mapbox instance in container ref -> binds event listeners -> fetches project spatial data -> renders geometries -> handles updates -> cleans up map instance on unmount.
- Potential Bugs: Canvas sizing collapses to zero dimensions if parent flexbox container styles are misconfigured; memory leaks occur if map.remove() is omitted on unmount.
- Modern Reconstruction Strategy: Wrap Mapbox logic in a custom hook (useMapbox) to isolate imperative canvas manipulation from React declarative UI components.

6. Toolbar Component

- Purpose: Dual-sided floating interface providing direct user control over drawing instruments, environmental markers, viewing perspectives, and constraint modal toggles.
- User Interaction: Clicking SVG tool buttons (Boundary Polygon, Non-Developable Polygon, Tree Marker, Measure, Grid Toggle, 3D View, Reset Camera, Image Upload, Form Toggles).
- State Variables: activeButton (string), isUploadModalOpen (boolean), toolActivationRules (derived state object determining enabled/disabled states).
- Props Supported: currentBoundary (object/boolean), activeTool (string), onSelectTool (function), onToggleOverlay (function), projectId (string).
- Events Handled: Button onClick, file input onChange for image upload.
- API Calls: POST /api/projects/:id/upload-image (multipart/form-data) via file upload modal.
- Data Received: Image URL confirmation upon successful upload.
- Data Sent: FormData object containing binary image payload.
- UI States: Buttons display default, active (highlighted), disabled (muted opacity with pointer-events disabled), and modal open states.
- Loading States: Spinner within upload button during image file transmission.
- Error States: Disabled button tooltips explaining prerequisite actions (e.g., "Draw site boundary first to enable trees").
- Success States: Visual highlight toggled on active tool; file upload completion toast.
- Dependencies: React, CSS Modules (dynamic button classes), SVG Icon libraries.
- Relationship with Other Components: Child of MapPage; drives MapboxDraw modes and triggers constraint form overlays.
- Likely Lifecycle: Re-renders reactively as currentBoundary or activeTool props change, recalculating toolActivationRules.
- Potential Bugs: Attempting to activate drawing modes when Mapbox Draw has not finished initializing, throwing unhandled JavaScript null references.
- Modern Reconstruction Strategy: Model toolbar actions using an explicit state machine (such as XState or a localized reducer) with centralized keyboard shortcut bindings.

7. GetRegulatoryForm Component

- Purpose: Modal overlay allowing users to input and edit site-specific statutory parameters governing the parcel under municipal regulations.
- User Interaction: Entering road widths, choosing setback rules, specifying permissible building height constraints, submitting form.
- State Variables: roadWidth (number/string), frontSetback (number), rearSetback

- (number), sideSetback (number), heightCap (number), vali[span_24](start_span)[span_24](end_span)dationErrors (object).
- Props Supported: isOpen (boolean), onClose (function), projectId (string), initialData (object), onSave (function).
- Events Handled: Input onChange, form onSubmit, backdrop onClick.
- API Calls: POST /api/projects/:id/regulatory-constraints.
- Data Received: Confirmed regulatory schema object.
- Data Sent: Serialized regulatory constraint parameters.
- UI States: Centered modal with semi-transparent backdrop, responsive input grids, error messages.
- Loading States: Submit button loading spinner during network transmission.
- Error States: Inline validation errors for non-numeric or impossible dimensions (e.g., negative setback distances).
- Success States: Form close transition, success toast notification, propagation of updated constraints to parent state.
- Dependencies: React, Axios, React Toastify, CSS Modules.
- Relationship with Other Components: Rendered conditionally as an overlay inside MapPage; feeds constraints into InputSummary.
- Likely Lifecycle: Mounts on modal open -> populates from initialData -> updates local state -> submits via API -> updates parent -> unmounts on close.
- Potential Bugs: String values from numeric inputs (e.target.value) passed directly to calculations without parsing via parseFloat, resulting in string concatenation bugs.
- Modern Reconstruction Strategy: Use native HTML5 numeric constraints (min, step) combined with a typed schema validator to ensure float serialization.

8. BusinessConstraints Component

- Purpose: Interface for capturing commercial objectives, product typologies, and financial targets for the parcel development.
- User Interaction: Checking product type checkboxes (Residential, Retail, Commercial), selecting parking typologies (Basement vs Podium), specifying unit area targets.
- State Variables: prod[span_68](start_span)[span_68](end_span)[span_72](start_span)[span_72](end_span)uctMix (array of selected strings), parkingType (string), targetCarpetArea (number/string), formErrors (object), isSubmitting (boolean).
- Props Supported: isOp[span_69](start_span)[span_69](end_span)[span_73](start_span)[span_73](end_span)en (boolean), onClose (function), projectId (string), onSuccess (function).
- Events Handled: Checkbox toggle onChange, radio button selection, form submission.
- API Calls: POST /api/projects/:id/business-constraints.
- Data Received: Persisted business constraint record.
- Data Sent: JSON payload containing commercial parameters.
- UI States: Structured multi-section form with checkbox groups, segmented radio controls, and numeric inputs.
- Loading States: Loading indicator locking the submission controls during processing.
- Error States: Validation flags if zero product typologies are selected or target metrics violate basic thresholds.
- Success States: Data saved; automatically opens InputSummary or returns to map

workspace.

- Dependencies: React, Axios, React Toastify.
- Relationship with Other Components: Complements GetRegulatoryForm; feeds data directly into InputSummary and downstream financial models.
- Likely Lifecycle: Initialized -> captures commercial parameters -> persists to backend -> triggers transition to summary review.
- Potential Bugs: Desynchronization between array state mutations (e.g., mutating productMix directly instead of returning a new array reference), breaking re-renders.
- Modern Reconstruction Strategy: Use immutability utilities or functional updater forms within useState to guarantee pure array operations.

9. GetBusinessConstraints Component

- Purpose: Read-only or fetch-and-edit overlay component designed to retrieve and display previously saved business constraints from the backend.
- User Interaction: Viewing existing constraint configurations, clicking "Edit" to unlock form fields, clicking "Close".
- State Variables: constraintsData (object), isLoading (boolean), isEditing (boolean), fetchError (string).
- Props Supported: projectId (string), isOpen (boolean), onClose (function).
- Events Handled: Fetch execution on mount/open, toggle edit mode onClick.
- API Calls: GET /api/projects/:id/business-constraints.
- Data Received: Previously saved business constraint entity.
- Data Sent: None (read mode) or updated JSON payload (edit mode).
- UI States: Display cards showing selected parameters; editable input fields when edit mode is toggled.
- Loading States: Centered spinning loader while retrieving remote records.
- Error States: Error alert banner with retry button if backend retrieval fails.
- Success States: Smooth rendering of structured constraint details.
- Dependencies: React, Axios, React Toastify.
- Relationship with Other Components: Intermediary between raw constraint entry (BusinessConstraints) and final verification (InputSummary).
- Likely Lifecycle: Mounts -> fires GET request -> populates display -> switches to edit if requested -> unmounts.
- Potential Bugs: Race conditions occurring if the user rapidly opens and closes the overlay before the GET promise resolves.
- Modern Reconstruction Strategy: Implement AbortController signal injection inside the Axios call to cancel in-flight requests upon unmount.

10. InputSummary Component

- Purpose: Pre-flight review overlay displaying a consolidated summary of site boundaries, environmental deductions, regulatory constraints, and business targets.
- User Interaction: Reviewing tabular data, clicking "Back to Edit", clicking "Save and Continue" to navigate downstream.
- State Variables: isConfirming (boolean), summaryData (consolidated object).
- Props Supported: projectId (string), isOpen (boolean), onClose (function), onContinue (function).

- Events Handled: Action button onClick, keyboard ESC to close.
- API Calls: POST /api/projects/:id/finalize-inputs or PUT /api/projects/:id confirming feasibility input completeness.
- Data Received: Full aggregated project entity.
- Data Sent: Confirmation status flag.
- UI States: Multi-column structured data table, status badges, fixed footer with primary action buttons.
- Loading States: Full-overlay loading spinner during downstream workflow transition.
- Error States: Toast notifications indicating missing mandatory parameters (e.g., "Cannot proceed: Site boundary has not been drawn").
- Success States: Clean programmatic transition to /site-constraints via React Router.
- Dependencies: React, React Router (useNavigate), React Toastify, CSS Modules.
- Relationship with Other Components: The final gatekeeper component on MapPage; unlocks subsequent generative layout pipelines.
- Likely Lifecycle: Mounts -> aggregates state from project context or API -> displays review matrix -> executes final save -> navigates to /site-constraints.
- Potential Bugs: Navigating to /site-constraints without verifying that the spatial geometries have finished persisting to the database, resulting in downstream 404/empty canvas errors.
- Modern Reconstruction Strategy: Implement a strict server-side readiness check before navigation, returning explicit validation tokens.

Part 4: React Technical Foundations for Geospatial Applications

Developing geospatial interfaces requires deep technical mastery of React hooks, component composition, and memory lifecycle management. Because WebGL maps maintain continuous imperative rendering loops outside of standard React virtual DOM reconciliation, architectural boundaries must be enforced.

Core Conceptual Matrix

React Concept	General Meaning	Application in Planwise	Common Implementation Pitfalls
Components	Independent, reusable UI units returning JSX markup.	Isolating the Map container, toolbar buttons, project tiles, and modal forms.	Creating monolith components where map canvas logic and form state are coupled.
JSX	Syntax extension allowing HTML-like structures inside JavaScript.	Declaring dynamic DOM structures for HUD area readouts and SVG icons.	Embedding complex coordinate transformation logic directly within JSX expressions.
Props	Immutable data passed from parent to child components.	Passing active boundary statuses to Toolbar to trigger tool	Mutating props directly or excessive prop drilling across deeply

React Concept	General Meaning	Application in Planwise	Common Implementation Pitfalls
		activation rules.	nested modal components.
useState	Hook declaring reactive state variables triggering re-renders.	Managing form inputs, modal visibility, and area metric readouts.	Storing the Mapbox map instance in useState, causing continuous canvas re-instantiations.
useEffect	Hook executing side-effects synchronized with dependency updates.	Initializing Mapbox canvas on mount, fetching project catalogs, handling cleanup.	Omitting cleanup functions, leading to duplicate event listeners or memory leaks on unmount.
useRef	Hook persisting mutable values across renders without causing re-renders.	Holding the Mapbox Map instance, Mapbox Draw object, and auto-save timer handles.	Attempting to use ref changes to drive visual DOM updates without companion state variables.
Controlled Inputs	Form elements whose values are bound to and driven by React state.	Binding project title, description, and land ownership dropdown values.	Forgetting onChange handlers, locking the inputs and preventing user typing.
Conditional Rendering	Dynamically rendering UI elements based on state conditions.	Toggling regulatory form overlays, error messages, and loading spinners.	Using && with numeric zero (0 && <Component />), resulting in a literal "0" printed to UI.
Component Composition	Nesting specialized components inside generalized layout containers.	Wrapping the interactive map inside a layout containing floating toolbars and modals.	Inflexible hardcoding of overlays inside child components rather than using composition.
Protected Routes	Route wrappers evaluating authentication state before rendering paths.	Restricting /home, /map, and /site-constraints to users with valid tokens.	Flashing protected content before authentication checks complete, exposing UI shells.

Technical Analysis with Representative Implementations

1. Managing Mapbox Instances via useRef

A Mapbox GL JS map instance manages its own internal WebGL rendering loop, DOM canvas nodes, and animation frames. If a developer stores the map instance within a useState hook, every subsequent render triggers shallow reference comparisons that can cause unstable component re-renders, or re-instantiate the entire WebGL canvas. To prevent this, the map instance must be preserved in a useRef.

```

// Representative reconstruction based on documented behavior
import React, { useEffect, useRef, useState } from 'react';
import mapboxgl from 'mapbox-gl';
import 'mapbox-gl/dist/mapbox-gl.css';

export function MapContainer({ onMapReady }) {
  const mapContainerRef = useRef(null);
  const mapInstanceRef = useRef(null);
  const [mapError, setMapError] = useState(null);

  useEffect(() => {
    if (mapInstanceRef.current) return;

    try {
      mapboxgl.accessToken = process.env.REACT_APP_MAPBOX_TOKEN;

      const map = new mapboxgl.Map({
        container: mapContainerRef.current,
        style: 'mapbox://styles/mapbox/satellite-streets-v12',
        center: [72.8777, 19.0760],
        zoom: 13,
      });

      map.on('load', () => {
        mapInstanceRef.current = map;
        if (onMapReady) onMapReady(map);
      });
    } catch (err) {
      setMapError('Failed to initialize WebGL map canvas.');
    }

    return () => {
      if (mapInstanceRef.current) {
        mapInstanceRef.current.remove();
        mapInstanceRef.current = null;
      }
    };
  }, [onMapReady]);

  if (mapError) {
    return <div className="map-error-fallback">{mapError}</div>;
  }

  return <div ref={mapContainerRef}
    className="mapbox-gl-canvas-container" />;
}

```

A common failure mode is omitting the invocation of `map.remove()` within the cleanup return of

the effect hook. In single-page architectures where users navigate between the dashboard and map views, each mount creates an active WebGL context in browser memory. Once the browser reaches its hardware context limit—typically between 8 and 16 concurrent contexts—the graphics subsystem crashes with an unrecoverable `CONTEXT_LOST_WEBGL` exception.

2. Controlled Multi-Attribute Form Management

Capturing project initialization parameters requires robust, controlled inputs where individual keystrokes validate against business constraints.

```
// Representative reconstruction based on documented behavior
import React, { useState } from 'react';
import axios from 'axios';
import { toast } from 'react-toastify';

const OWNERSHIP_OPTIONS = [
  "Private Land (Freehold/Leasehold)",
  "Collector Land (Class II)",
  "Collector Land (Leasehold)",
  "Estate Land (Leasehold)",
  "MHADA Land"
];

export function ControlledNewProjectForm({ onSuccess }) {
  const [formData, setFormData] = useState({
    projectTitle: '',
    projectDescription: '',
    landOwnership: OWNERSHIP_OPTIONS[0]
  });
  const [errors, setErrors] = useState({});
  const [isSubmitting, setIsSubmitting] = useState(false);

  const validate = () => {
    const errs = {};
    if (!formData.projectTitle.trim()) errs.projectTitle = 'Project title is required.';
    if (!formData.projectDescription.trim()) errs.projectDescription = 'Description is required.';
    if (!formData.landOwnership) errs.landOwnership = 'Ownership type is required.';
    return errs;
  };

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value }));
    if (errors[name]) {
      setErrors(prev => ({ ...prev, [name]: null }));
    }
  };
}
```

```

    }
  };

const handleSubmit = async (e) => {
  e.preventDefault();
  const validationErrors = validate();
  if (Object.keys(validationErrors).length > 0) {
    setErrors(validationErrors);
    return;
  }

  setIsSubmitting(true);
  try {
    const token = localStorage.getItem('token');
    const payload = {
      project_title: formData.projectTitle,
      about: formData.projectDescription,
      land_ownership: formData.landOwnership,
      geojson: "{}",
      nondevelopableareas: "[]",
      trees: "[]"
    };

    const response = await
    axios.post('http://localhost:5001/api/projects', payload, {
      headers: { Authorization: `Bearer ${token}` }
    });

    if (response.status === 201) {
      toast.success('Project created successfully.');
      onSuccess(response.data);
    }
  } catch (err) {
    toast.error(err.response?.data?.message || 'Server error
creating project. ');
  } finally {
    setIsSubmitting(false);
  }
};

return (
  <form onSubmit={handleSubmit} className="project-form">
    <div className="form-group">
      <label>Project Title</label>
      <input
        name="projectTitle"
        value={formData.projectTitle}
        onChange={handleChange}

```

```

        className={errors.projectTitle ? 'input-error' : ''}
        disabled={isSubmitting}
      />
      {errors.projectTitle && <span
className="error-text">{errors.projectTitle}</span>}
    </div>

    <div className="form-group">
      <label>Land Ownership</label>
      <select
        name="landOwnership"
        value={formData.landOwnership}
        onChange={handleChange}
        disabled={isSubmitting}
      >
        {OWNERSHIP_OPTIONS.map(opt => (
          <option key={opt} value={opt}>{opt}</option>
        ))}
      </select>
    </div>

    <button type="submit" disabled={isSubmitting}>
      {isSubmitting ? 'Creating Project...' : 'Create Project'}
    </button>
  </form>
);
}

```

A frequent error in form architecture is direct in-place mutation of state properties, such as assigning `formData.projectTitle = e.target.value`. Because React evaluates state changes via shallow reference checks, direct mutation bypasses the reconciler, preventing the user interface from reflecting typed inputs.

Part 5: Authentication and JWT Lifecycle

JSON Web Token (JWT) Fundamentals

A JSON Web Token (RFC 7946/RFC 7519) represents claims securely transferred between two parties. It comprises three base64url-encoded segments joined by periods:

1. Header: Specifies the hashing algorithm, such as HMAC SHA-256.
2. Payload: Encloses the claim set, including user identifiers, role definitions, and expiration timestamps (exp).
3. Signature: Computed by hashing the header and payload using a server-side secret, enabling verification of transmission integrity.

Planwise employs stateless JWT tokens to eliminate server-side session stores, allowing the REST service to verify user requests independently by inspecting the signature.

Documented Authentication Flow

The internship report documents a complete client-side authentication lifecycle:

- The user inputs credentials into the Login component.
- The client transmits an asynchronous POST request with credentials to the authentication endpoint.
- The server validates credentials against storage and issues an encrypted JWT.
- The client receives the JWT and writes it to browser localStorage under the key 'token'.
- On subsequent routing attempts, the <ProtectedRoute> wrapper queries localStorage. If the token exists, navigation proceeds; if absent, the route redirects to /login.
- Outgoing API calls retrieve the token from localStorage and inject it into the HTTP headers as Authorization: Bearer <token>.
- The backend validates the signature, extracts the user context, and fulfills the resource request.

Protected Route Architecture

The protected route acts as a client-side access control proxy.

```
// Representative reconstruction based on documented behavior
import React from 'react';
import { Navigate, useLocation } from 'react-router-dom';

export function ProtectedRoute({ children }) {
  const location = useLocation();
  const token = localStorage.getItem('token');

  if (!token) {
    return <Navigate to="/login" state={{ from: location }} replace
/>;
  }

  return children;
}
```

Security Considerations: LocalStorage Versus HttpOnly Cookies

The documented implementation relies on localStorage to persist authentication tokens. While functional for rapid single-page application development, it has clear security tradeoffs:

Evaluation Dimension	Browser localStorage (Documented)	Secure HttpOnly Cookies (Enterprise Production Alternative)
XSS Vulnerability	High: Any malicious script injected via third-party dependencies can execute <code>localStorage.getItem('token')</code> and exfiltrate the credential.	Immune: JavaScript code executed in the browser cannot read the contents of an HttpOnly cookie.

Evaluation Dimension	Browser localStorage (Documented)	Secure HttpOnly Cookies (Enterprise Production Alternative)
CSRF Vulnerability	Immune: The token must be explicitly injected via JavaScript into the Authorization header.	Vulnerable: Browsers automatically attach cookies to cross-site requests; requires Anti-CSRF tokens or SameSite=Strict flags.
Implementation Simplicity	Simple: Requires zero complex domain/CORS cookie configurations during local multi-port development (localhost:3000 to localhost:5001).	Moderate: Requires explicit CORS credentials configuration (withCredentials: true), secure flags, and domain matching.
Session Invalidation	Client-side only: Clearing localStorage removes the token locally, but the token remains cryptographically valid on the server until expiration.	Server-controlled: Server can drop refresh sessions in a database or revoke cookies instantly.

For an enterprise deployment handling sensitive commercial land acquisition data, authentication should transition to a dual-token strategy: an ephemeral, memory-only access token (held in React state) refreshed silently via an encrypted, HttpOnly, SameSite=Strict refresh cookie managed by the backend gateway.

Part 6: Mapbox Deep Dive

Mapbox GL JS Mechanics

Mapbox GL JS is an interactive mapping library powered by WebGL. Unlike traditional tile maps that render static raster image squares (.png/.jpg), Mapbox renders vector tiles dynamically on the client's GPU. Mapbox GL JS parses mathematical vector descriptions of geographic roads, contours, and boundaries, styling them in real time at 60 frames per second according to a style JSON specification.

The mapping subsystem is organized around several core primitives:

- **Map Instance:** The root controller orchestrating camera perspective, viewport dimensions, user interactions, and the rendering loop.
- **Sources:** Spatial data providers that deliver raw geometries (e.g., GeoJSON, Vector Tile endpoints, Raster DEMs) independent of their visual styling.
- **Layers:** Visual styling rules tied to a source, defining whether geometries are rendered as colored fills, lines, extrusions, or symbols.
- **Controls:** UI plugins injected into the map container, such as zoom, compass navigation, and geocoding search bars.
- **Camera System:** Coordinates the 3D viewing perspective via spherical center coordinates, zoom levels, bearing rotation, and pitch tilt.

Mapbox Draw Integration and Event Model

Planwise integrates the `@mapbox/mapbox-gl-draw` library to allow users to digitize parcel perimeters directly onto the vector basemap. Mapbox Draw introduces operational editing modes:

- `simple_select`: Allows selecting, dragging, and deleting existing geometries.
- `direct_select`: Allows selecting and moving individual vertices or midpoints of a polygon.
- `draw_polygon`: Captures sequential mouse clicks to construct closed polygonal rings.
- `draw_point`: Captures a single click coordinate to instantiate point markers (used for tree positions).

The application attaches event listeners directly to the map instance to maintain synchronization between the imperative drawing canvas and React state:

- `draw.create`: Fires when a user completes a geometric shape (e.g., double-clicking the final vertex of a polygon).
- `draw.update`: Fires when vertices are modified, moved, or deleted in `direct_select` mode.
- `draw.delete`: Fires when an active geometry is removed via keyboard delete or programmatic call.
- `draw.modechange`: Fires when switching operational modes (e.g., from navigating to drawing).

```
// Representative reconstruction based on documented behavior
import MapboxDraw from '@mapbox/mapbox-gl-draw';
import '@mapbox/mapbox-gl-draw/dist/mapbox-gl-draw.css';

export function initializeMapboxDraw(map, onGeometryChange) {
  const draw = new MapboxDraw({
    displayControlsDefault: false,
    controls: {
      polygon: false,
      trash: false
    },
    defaultMode: 'simple_select'
  });

  map.addControl(draw);

  const handleUpdate = () => {
    const data = draw.getAll();
    onGeometryChange(data);
  };

  map.on('draw.create', handleUpdate);
  map.on('draw.update', handleUpdate);
  map.on('draw.delete', handleUpdate);

  return draw;
}
```

Custom Controls and Tools

The internship report documents several custom spatial controls integrated into the MapPage and Toolbar:

- **3D View Toggle:** Toggles map pitch dynamically between 0° (planimetric orthogonal view) and 60° (isometric view) to inspect topographical massing:

```
const toggle3DView = (map, is3D) => {  
  map.easeTo({  
    pitch: is3D ? 60 : 0,  
    bearing: is3D ? -20 : 0,  
    duration: 1000  
  });  
};
```
- **Measurement Tool:** Tracks distance along polyline segments or calculates bounding dimensions across polygon edges.
- **Grid Overlay:** Adds a procedural vector source generating equidistant coordinate lines across the parcel to evaluate structural spacing.
- **Mapbox Geocoder Restricted to India:** Restricts autocomplete address queries specifically to Indian territorial bounds using the bounding box parameter (bbox) or country code filter (countries: 'in').

Part 7: GeoJSON Masterclass

The RFC 7946 Specification

GeoJSON is an open standard format designed for representing simple geographical features along with their non-spatial attributes using JSON. The specification (RFC 7946) mandates the WGS84 coordinate reference system (EPSG:4326), with coordinates expressed in decimal degrees. Coordinate ordering must strictly follow [Longitude, Latitude] (easting, northing), which aligns with Cartesian mathematical convention ([X, Y]) but inverts colloquial speech ([Latitude, Longitude]).

GeoJSON Geometry Primitives

GeoJSON organizes geospatial data into explicit primitives:

Primitive Type	Structural Definition	Planwise Domain Entity	Structural Example
Point	A single coordinate array [X, Y] representing a specific geographic location.	Preserved Tree Marker: Pinpoints the trunk location of a tree.	{ "type": "Point", "coordinates": [72.8777, 19.0760] }
LineString	An array of two or more coordinate points forming continuous line segments.	Measurement Tool: Represents linear distances along setbacks or road widths.	{ "type": "LineString", "coordinates": [[72.87, 19.07], [72.88, 19.07]] }
Polygon	An array of Linear Rings (closed arrays of ≥ 4 coordinate points).	Site Boundary / Non-Developable Area: Delineates outer	{ "type": "Polygon", "coordinates": [[[72.87, 19.07], [72.88, 19.07],

Primitive Type	Structural Definition	Planwise Domain Entity	Structural Example
	where index [0] equals index [N]). The first ring is exterior; subsequent rings define interior "holes".	property perimeters or interior reservations.	[72.88, 19.08], [72.87, 19.07]]]]}
MultiPolygon	An array of Polygon coordinate structures representing disconnected parcels under a single entity.	Non-Contiguous Parcels: Joint-venture plots separated by public roads.	{"type": "MultiPolygon", "coordinates": [[[[...]]], [[[[...]]]]}
Feature	Wraps a geometry object with an identifier and an arbitrary dictionary of properties.	Annotated Element: Stores metadata like { "tree_species": "Banyan", "canopy_radius": 5 }.	{"type": "Feature", "geometry": {...}, "properties": {...}}
FeatureCollection	A top-level container holding a list of heterogeneous Feature objects.	Consolidated Project Payload: Encapsulates all boundaries, cutouts, and trees.	{"type": "FeatureCollection", "features": [...]}

Structural Mapping to Planwise Entities

In the Planwise architecture, spatial data models map directly to RFC 7946 structures:

- Site Boundary Polygon: Encapsulates the gross plot area.
- Non-Developable Area (Deductions): Modeled as distinct polygon features within the "nondevelopableareas" payload or as interior linear rings (holes) within the main boundary polygon, subtracting area from net calculations.
- Trees: Captured as point features within the "trees" payload. Each point marks a tree trunk requiring a radial preservation buffer during downstream massing.

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "id": "boundary_001",
      "properties": { "entity_type": "site_boundary", "area_sqm":
4046.86 },
      "geometry": {
        "type": "Polygon",
        "coordinates": [
          [
            [72.851234, 19.054321],
            [72.853456, 19.054321],
            [72.853456, 19.056543],
            [72.851234, 19.056543],
            [72.851234, 19.054321]
          ]
        ]
      }
    }
  ]
}
```



```

    ]
  },
  {
    "type": "Feature",
    "id": "tree_001",
    "properties": { "entity_type": "tree_marker", "diameter_meters":
1.2 },
    "geometry": {
      "type": "Point",
      "coordinates": [72.852100, 19.055100]
    }
  }
]
}

```

Spatial Validation and Geometric Integrity

Raw vector input from browser pointer interactions can easily introduce invalid geometric configurations. A robust frontend must validate coordinate structures before passing them to auto-save pipelines:

- Linear Ring Closure: The first coordinate pair and final coordinate pair of every polygon ring must be identical.
- Self-Intersection Prevention: Polygons must not cross their own edges (creating "figure-eight" or "bowtie" topologies). Self-intersecting polygons produce undefined area calculations in standard spatial databases.
- Exterior Enclosure: All interior deductions (non-developable polygons) and tree point coordinates must reside completely within the bounding perimeter of the primary site boundary polygon.
- Winding Order: Exterior rings must follow a counter-clockwise orientation, and interior rings must follow a clockwise orientation under RFC 7946 conventions.

Part 8: High-Frequency Spatial Mutation and Auto-Save

The High-Frequency Mutation Challenge

When a user modifies a parcel boundary by dragging a vertex across the Mapbox canvas, the `draw.update` event fires continuously on every mouse movement frame—generating 60 coordinate updates per second. Dispatching an HTTP request on every event causes backend connection pool exhaustion, saturates browser concurrent HTTP/1.1 connections, and creates asynchronous race conditions where earlier, slower requests arrive out of order and overwrite newer geometry states.

Throttling limits function execution to a steady rate (e.g., executing once every 100 milliseconds), making it ideal for updating the visible on-screen HUD metrics panel during

drawing. Debouncing postpones function execution until a specified quiet period has elapsed without any new invocations. Every new event resets the countdown timer. Debouncing is ideal for persistent auto-save mechanisms, ensuring that network operations execute only when the user finishes adjusting a shape.

Auto-Save Mechanics

The auto-save subsystem uses a React `useRef` to hold the debounce timer handle across renders, ensuring that new state captures cancel preceding scheduled requests. In addition, it employs an `AbortController` to cancel in-flight HTTP requests if a new save occurs before the previous network call finishes.

```
// Representative reconstruction based on documented behavior
import { useRef, useCallback } from 'react';
import axios from 'axios';
import { toast } from 'react-toastify';

export function useDebouncedMapSave(projectId, delay = 1000) {
  const timeoutRef = useRef(null);
  const abortControllerRef = useRef(null);

  const triggerAutoSave = useCallback((geojsonPayload) => {
    if (timeoutRef.current) {
      clearTimeout(timeoutRef.current);
    }

    timeoutRef.current = setTimeout(async () => {
      if (abortControllerRef.current) {
        abortControllerRef.current.abort();
      }
      abortControllerRef.current = new AbortController();

      try {
        const token = localStorage.getItem('token');
        await axios.put(
          `http://localhost:5001/api/projects/${projectId}`,
          geojsonPayload,
          {
            headers: { Authorization: `Bearer ${token}` },
            signal: abortControllerRef.current.signal
          }
        );
        toast.success('Map changes auto-saved.', { autoClose: 1500 });
      } catch (err) {
        if (!axios.isCancel(err)) {
          toast.error('Failed to auto-save map changes.');
        }
      }
    }, delay);
  }, delay);
```

```

    }, [projectId, delay]));

    return { triggerAutoSave };
}

```

Part 9: Toolbar State Machine and Dynamic Guard Rules

Dynamic Toolbar Activation Rules

The Planwise interface enforces strict operational constraints: users cannot add site features (e.g., placing tree markers or non-developable areas) until a valid primary parcel boundary has been created.

Managing this conditional logic through scattered boolean expressions across various buttons leads to brittle code and invalid visual states. The system structures this through a Finite State Machine (FSM) driven by a centralized toolActivationRules object.

Current System State	Triggering Event	Target System State	Guard Condition / Prerequisite	Enabled Tools	Disabled Tools
NO_PROJECT	SELECT_PROJECT	PROJECT_ACTIVE_NO_BOUNDARY	Project record exists with valid ID.	Location Search, View Reset.	Boundary Draw, Tree Draw, Non-Developable Draw, Constraints.
PROJECT_ACTIVE_NO_BOUNDARY	DRAW_BOUNDARY_COMPLETE	BOUNDARY_DEFINED	Completed polygon ring with >= 3 unique vertices.	Boundary Draw, Delete Boundary, Location Search, 3D Toggle.	Tree Marker Tool, Non-Developable Area Tool, Constraint Overlays.
BOUNDARY_DEFINED	SELECT_TREE_TOOL	PLACING_TREES	Active boundary polygon exists in Mapbox Draw.	Tree Marker Tool, Simple Select, Delete.	Regulatory Forms, Downstream Continuations.
BOUNDARY_DEFINED	SELECT_DEDUCTION_TOOL	DRAWING_DEDUCTION	Active boundary polygon exists in Mapbox Draw.	Non-Developable Draw, Delete.	Tree Marker Tool, Regulatory Forms.
BOUNDARY_DEFINED	OPEN_REGULATORY	EDITING_CONSTRAINTS	Primary boundary defined and validated.	Form Inputs, Save Actions.	All Drawing Canvas Tools.

Current System State	Triggering Event	Target System State	Guard Condition / Prerequisite	Enabled Tools	Disabled Tools
CONSTRAINT S_REVIEWED	CLICK_CONTINUE	TRANSITION_NEXT_STEP	Boundary, non-developable areas, and all required forms completed.	"Save and Continue" Action.	In-flight editing tools.

Declarative React Implementation

The dynamic state machine is implemented by separating derived state rules from visual icon rendering.

```
// Representative reconstruction based on documented behavior
import React, { useMemo } from 'react';
```

```
export function Toolbar({
  hasBoundary,
  activeTool,
  onSelectTool,
  onToggleModal,
  isSaving
}) {
  const toolRules = useMemo(() => ({
    canDrawBoundary: true,
    canPlaceTrees: Boolean(hasBoundary),
    canDrawDeductions: Boolean(hasBoundary),
    canOpenConstraints: Boolean(hasBoundary),
    canOpenSummary: Boolean(hasBoundary),
  })), [hasBoundary]);

  return (
    <aside className="toolbar-container">
      <div className="toolbar-group left-tools">
        <button
          className={`toolbar-btn ${activeTool === 'boundary' ?
'active' : ''}`}
          onClick={() => onSelectTool('boundary')}
          title="Draw Site Boundary"
        >
          <svg className="tool-icon">{/* Boundary Polygon SVG
*/}</svg>
        </button>

        <button
          className={`toolbar-btn ${activeTool === 'tree' ? 'active' :
''} ${!toolRules.canPlaceTrees ? 'toolbar-btn--disabled' : ''}`}
          onClick={() => toolRules.canPlaceTrees &&
```

```

onSelectTool('tree'){
    disabled={!toolRules.canPlaceTrees}
    title={toolRules.canPlaceTrees ? "Place Tree Markers" :
"Draw boundary first to place trees"}
    >
    <svg className="tool-icon">{/* Tree Marker SVG */}</svg>
</button>

    <button
        className={`toolbar-btn ${activeTool === 'deduction' ?
'active' : ''} ${!toolRules.canDrawDeductions ?
'toolbar-btn--disabled' : ''}`}
        onClick={() => toolRules.canDrawDeductions &&
onSelectTool('deduction')}
        disabled={!toolRules.canDrawDeductions}
        title={toolRules.canDrawDeductions ? "Draw Non-Developable
Area" : "Draw boundary first"}
    >
        <svg className="tool-icon">{/* Cutout Polygon SVG */}</svg>
    </button>
</div>

<div className="toolbar-group right-tools">
    <button
        className={`toolbar-btn ${!toolRules.canOpenConstraints ?
'toolbar-btn--disabled' : ''}`}
        onClick={() => toolRules.canOpenConstraints &&
onToggleModal('regulatory')}
        disabled={!toolRules.canOpenConstraints}
        title="Open Regulatory Constraints"
    >
        <svg className="tool-icon">{/* Regulatory Document SVG
*/}</svg>
    </button>

    <button
        className={`toolbar-btn ${!toolRules.canOpenConstraints ?
'toolbar-btn--disabled' : ''}`}
        onClick={() => toolRules.canOpenConstraints &&
onToggleModal('business')}
        disabled={!toolRules.canOpenConstraints}
        title="Open Business Constraints"
    >
        <svg className="tool-icon">{/* Business Chart SVG */}</svg>
    </button>
</div>
</aside>
);

```

```
}
```

Part 10: Form State Architecture and Multi-Tier Validation

Form Lifecycles in the Planwise Workflow

Planwise uses forms for project creation (NewProject), municipal constraint capture (GetRegulatoryForm), commercial target definitions (BusinessConstraints), and summary reviews (InputSummary). A form transitions from a pristine state to an editing state upon user interaction, where field-level inputs update local state. When submission is triggered, the client executes validation logic. If validation fails, error states highlight invalid fields and render descriptive alerts. If validation succeeds, the form enters a submitting state where inputs are locked and buttons display spinners. Upon receiving an HTTP 2xx response, the form displays confirmation feedback, resets its fields, and closes the modal; if the server rejects the request, an error state displays API feedback without clearing the user's inputs.

Validation operates across multiple tiers:

1. Immediate UI Validation (Client-Side): Executes synchronously on input change or blur, checking string lengths, numeric formats, and required fields.
2. Pre-Flight Boundary Validation (Client-Side): Checks geometric validity before submission, such as verifying that setback distances do not exceed total parcel dimensions.
3. Statutory Schema Validation (Server-Side): The backend validates incoming payloads against municipal building codes (such as DCPR 2034 or UDCPR) and database integrity constraints, ensuring client-side checks cannot be bypassed.

```
// Representative reconstruction based on documented behavior
export function validateRegulatoryInputs(values, siteMetrics) {
  const errors = {};

  const roadWidth = parseFloat(values.roadWidth);
  if (isNaN(roadWidth) || roadWidth <= 0) {
    errors.roadWidth = 'Road width must be a positive numeric value.';
  } else if (roadWidth < 6.0) {
    errors.roadWidth = 'Statutory minimum access road width is 6.0
meters.';
  }

  const frontSetback = parseFloat(values.frontSetback);
  if (isNaN(frontSetback) || frontSetback < 0) {
    errors.frontSetback = 'Front setback cannot be negative.';
  }

  if (siteMetrics && siteMetrics.parcelWidth) {
    const totalSetbacks = frontSetback + parseFloat(values.rearSetback
|| 0);
```

```

        if (totalSetbacks >= siteMetrics.parcelWidth) {
            errors.frontSetback = 'Total setbacks exceed parcel width; no
buildable envelope remains.';
        }
    }

    return {
        errors,
        isValid: Object.keys(errors).length === 0
    };
}

```

Part 11: API Specification and Backend Integration

The Project Creation Payload

The internship report documents the project creation contract. The request is sent via HTTP POST to `http://localhost:5001/api/projects`:

Request Headers:

`POST /api/projects HTTP/1.1`

`Host: localhost:5001`

`Content-Type: application/json`

`Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`

```

{
  "project_title": "Bandra Kurla Mixed Development",
  "about": "Site feasibility assessment for residential high-rise with
ground-floor commercial.",
  "land_ownership": "Private Land (Freehold/Leasehold)",
  "geojson": "{}",
  "nondevelopableareas": "[]",
  "trees": "[]"
}

```

Payload Schema Breakdown

The documented payload structure reveals several key architectural design choices:

- `project_title` (String): The administrative name of the development site. Used for search indexing and dashboard rendering.
- `about` (String): A narrative overview of the project brief, describing development objectives and scope.
- `land_ownership` (String): A categorized string identifying the land tenure type. This maps directly to one of the five documented legal categories (e.g., "MHADA Land", "Collector Land (Class II)"), which drive statutory FSI rules on the backend.
- `geojson` (Stringified JSON): Initialized as an empty stringified JSON object ("{}").

Represents the primary site boundary polygon. When populated, it stores the GeoJSON FeatureCollection or Polygon geometry defining the property perimeter.

- `nondevelopableareas` (Stringified Array): Initialized as an empty stringified array ("[]"). Stores an array of GeoJSON polygon features representing interior site exclusions (e.g., public reservations, utility easements).
- `trees` (Stringified Array): Initialized as an empty stringified array ("[]"). Stores an array of GeoJSON point features representing preserved tree trunk locations.

The backend schema accepts the spatial fields (`geojson`, `nondevelopableareas`, `trees`) as stringified JSON rather than raw nested JSON objects. The client must invoke `JSON.stringify()` on spatial objects before dispatching payloads.

Documented Endpoints Versus Inferred API Surface

HTTP Verb	Endpoint Path	Source Status	Purpose and Data Handling
POST	<code>/api/projects</code>	Explicitly Documented	Instantiates a new project record with title, ownership, and empty spatial geometries.
GET	<code>/api/projects</code>	Explicitly Documented	Fetches user projects; returns an array of parcel summaries for the dashboard.
DELETE	<code>/api/projects/:id</code>	Explicitly Documented	Removes a project and its associated spatial data by ID.
	POST	<code>/api/projects/:id/upload-image</code>	Explicitly Documented
PUT	<code>/api/projects/:id</code>	Reasonable Inference	Auto-save endpoint receiving updated GeoJSON boundaries, tree markers, and deductions.
POST	<code>/api/projects/:id/regulatory</code>	Reasonable Inference	Persists road widths, municipal setbacks, and height limits from <code>GetRegulatoryForm</code> .
POST	<code>/api/projects/:id/businesses</code>	Reasonable Inference	Persists product typologies, parking choices, and area targets from <code>BusinessConstraints</code> .
GET	<code>/api/projects/:id/businesses</code>	Reasonable Inference	Retrieves saved business constraints for display in <code>GetBusinessConstraints</code> .

HTTP Verb	Endpoint Path	Source Status	Purpose and Data Handling
POST	/api/auth/login	Reasonable Inference	Authenticates user credentials and issues the signed JSON Web Token.
Database Engine / Schema	<i>Not Documented</i>	Unknown in Source	Specific database engine (PostgreSQL/PostGIS, MongoDB, or Supabase) is not documented.

Part 12: Error Handling and User Experience

In complex spatial web applications, unhandled errors can crash the WebGL canvas, leaving users with a broken interface and risking unrecoverable data loss. Planwise implements a multi-layered defensive UI architecture to ensure reliability:

- Optimistic UI updates apply visual changes to the canvas immediately while mutations sync in the background.
- Client-side guard rails disable unavailable tools and display inline validation warnings before network requests are dispatched.
- Axios interceptors catch authentication errors and network failures globally, routing users to login or displaying retry prompts.
- React Toastify notifications provide non-intrusive status updates for background operations like auto-save.
- React Error Boundaries wrap the map canvas and modal overlays, containing component crashes and preventing the entire application from unmounting.
- The heads-up display (HUD) on the map interface continuously updates calculated gross and net areas as the user draws, providing immediate spatial feedback.

Part 13: Project Data Model

The conceptual data model represents the complete entities, relationships, and metadata structures documented in or reasonably inferred from the internship report.

The primary entity hierarchy begins with the User record, which owns multiple Project records. Each Project encapsulates administrative metadata (Title, About description, Creation timestamps), legal land ownership enums, binary preview images, and geographic geometry payloads (boundary polygons, deduction polygons, and tree point arrays). In addition, each Project maintains one-to-one relationships with statutory Regulatory Constraint records and commercial Business Constraint records.

Entity Attribute Specifications

1. Entity: User (Reasonably Inferred)

- id (String / UUID): Primary key uniquely identifying the registered user.
- email (String): Verified email address used for login and notifications.

- password_hash (String): Securely hashed password string (never exposed to client).
- created_at (Timestamp): Account registration timestamp.

2. Entity: Project (Documented)

- id (String / UUID): Primary key uniquely identifying the development project.
- user_id (String / UUID): Foreign key mapping the project to its owning user (Reasonably Inferred).
- project_title (String): Administrative title of the project.
- about (String): Narrative description of the site and development goals.
- land_ownership (Enum): Legal ownership classification:
 - "Private Land (Freehold/Leasehold)"
 - "Collector Land (Class II)"
 - "Collector Land (Leasehold)"
 - "Estate Land (Leasehold)"
 - "MHADA Land"
- geojson (String / GeoJSON Text): Primary parcel boundary polygon coordinates.
- nondevelopableareas (String / Array Text): Array of interior exclusion zone polygons.
- trees (String / Array Text): Array of preserved tree marker points.
- image_url (String, Optional): Path to project preview or satellite aerial imagery.
- created_at / updated_at (Timestamp): Record lifecycle timestamps (Reasonably Inferred).

3. Entity: RegulatoryConstraint (Documented)

- id (String / UUID): Unique identifier (Reasonably Inferred).
- project_id (String / UUID): Foreign key to associated Project entity.
- road_width_meters (Float): Width of adjacent primary access roadway (Reasonably Inferred).
- front_setback_meters (Float): Mandatory front boundary clearance offset (Reasonably Inferred).
- rear_setback_meters (Float): Mandatory rear boundary clearance offset (Reasonably Inferred).
- side_setbacks_meters (Float): Mandatory lateral property clearance offsets (Reasonably Inferred).
- height_limit_meters (Float, Optional): Maximum permissible vertical building cap (Reasonably Inferred).

4. Entity: BusinessConstraint (Documented)

- id (String / UUID): Unique identifier (Reasonably Inferred).
- project_id (String / UUID): Foreign key to associated Project entity.
- product_mix (Array of Strings): Selected typologies (e.g., ["Residential", "Retail"]).
- parking_typology (Enum): Parking design choice (e.g., "Basement", "Podium").
- target_carpet_area_sqm (Float, Optional): Target commercial saleable floor area (Reasonably Inferred).

Database Implementation Status

The exact underlying database management system (PostgreSQL with PostGIS, MongoDB, MySQL, or a BaaS platform like Supabase) is not documented in the supplied source report and remains an unknown implementation detail.

Part 14: End-to-End Execution Trace

To illustrate how data moves through every tier of the system, this trace follows a complete user journey: creating a new residential project on a parcel in Mumbai, digitizing its boundary, adding constraints, and generating the input summary.

Step #	User Action / Stage	UI / Component View	React State & Hook Updates	Events / Protocols	Data Structure / Payload	Persistence & Backend Operations	Error Handling & Guardrails
1	User enters email and password; clicks "SignIn".	Login component.	email, password, isLoading = true.	onSubmit -> Axios POST /api/auth/login.	{ email: "dev@archonet.in", password: "....." }.	Server verifies hash against database; issues signed JWT.	If 401: isLoading = false, renders error toast.
2	Authentication successful; receives token.	Dynamic redirect from /login to /home.	Navigated route changes; session initialized.	Window.localStorage.setItem('token', jwt).	JWT Bearer token string.	Token persisted locally in browser storage.	ProtectedRoute blocks access if token is missing.
3	Project dashboard loads.	Projects component at /home.	isLoading = true, visibleCount = 7.	Component mount -> useEffect fires Axios GET /api/projects.	Headers: { Authorization: "Bearer ..." }.	Database queries all projects belonging to user ID.	If network fails: displays error alert and empty list fallback.
4	Projects populate; user clicks "Create New Project".	Projects grid renders cards; NewProject modal opens.	isModalOpen = true in parent state.	onClick event on the creation card.	Component props pass modal toggle handlers.	No remote persistence at this stage.	Modal traps keyboard focus; closes on ESC key.
5	User inputs project metadata into modal.	NewProject modal form.	projectTitle = "Bandra Heights", landOwnership = "MHADA Land".	onChange events on controlled inputs and select dropdown.	Local React state object.	Ephemeral memory storage in component hook.	Dynamic validation clears errors as user types.
6	User clicks	NewProject	isSubmitting	onSubmit	Validates	None yet;	If invalid:

Step #	User Action / Stage	UI / Component View	React State & Hook Updates	Events / Protocols	Data Structure / Payload	Persistence & Backend Operations	Error Handling & Guardrails
	submit in modal.	modal form.	g = true, errors = {}.	fires handleSubmit.	title and ownership are non-empty.	runs client-side pre-flight checks.	blocks API call and highlights offending fields.
7	Client sends project creation payload.	NewProject modal form.	Inputs locked; submit button displays spinner.	Axios POST http://localhost:5001/api/projects.	{ project_title: "...", about: "...", land_ownership: "...", geojson: "{}", nondevelopableareas: "[]", trees: "[]" }.	API validates Bearer token; opens database transaction.	Catches network timeouts; unlocks inputs if connection drops.
8	Server confirms creation.	NewProject displays success alert.	success = "Project created successfully", isSubmitting = false.	API responds with HTTP 201 Created.	JSON response containing { id: "proj_99", ... }.	New project record written to database.	Validates returned payload contains valid entity ID.
9	User selects project; routes to workflow picker.	ProjectSelection component.	projectContext populated via location.state.	Programmatic navigation: navigate('/project-selection', { state: { projectId: "proj_99" } }).	Project context object containing ID and title.	None.	If refreshed without state: falls back to fetch via ID or redirects to /home.
10	User clicks "Site Feasibility" workflow card.	MapPage initializes.	mapLoaded = false, activeTool = null.	Programmatic navigation: navigate('/map', { state: { projectId: "proj_99" } }).	Project context passed along route.	None.	ProtectedRoute verifies JWT before allowing entry.
11	Mapbox	MapPage	mapLoaded	WebGL	Map style	None.	If WebGL

Step #	User Action / Stage	UI / Component / View	React State & Hook Updates	Events / Protocols	Data Structure / Payload	Persistence & Backend Operations	Error Handling & Guardrails
	canvas initializes.	fullscreen canvas.	d = true; useRef captures Map instance.	context created; Mapbox load event fires.	vector tiles loaded from Mapbox servers.		fails: unmounts canvas and displays fallback error message.
12	User searches "Bandra, Mumbai".	Mapbox Geocoder input overlay.	Camera coordinates update.	Geocoder autocomplete query; user selects result.	Bounding box coordinates restricted to India (countries: 'in').	Optional auto-save of parcel center coordinate.	Toast notification if geocoding service times out.
13	User activates polygon tool and draws perimeter.	Mapbox canvas in drawing mode.	activeTool = 'boundary', hasBoundary = true.	draw.create event fires on polygon completion.	GeoJSON Feature with Polygon geometry and 5 coordinate pairs.	State stored in Mapbox Draw internal memory.	Validates ring closure and checks for self-intersections.
14	System calculates gross area.	Heads-up Metrics Display HUD.	areaMetrics = { gross: 4046.86, net: 4046.86 }.	React state update triggers HUD re-render.	Numerical area value in square meters and square feet.	Ephemeral UI state.	Catches zero-area polygons and ignores calculations.
15	Toolbar enables environmental tools.	Toolbar component.	toolActivationRules.canPlaceTrees = true.	React re-render of toolbar with new props.	Boolean flags derived from hasBoundary.	None.	Guard conditions prevent activating tools without a boundary.
16	User places tree markers and marks non-developable area.	Mapbox canvas with tree and polygon overlays.	Coordinates appended to local tree and deduction arrays.	draw.create fires for each feature.	GeoJSON Point features for trees; Polygon feature for deduction.	Local memory within Mapbox Draw instance.	Boundary check ensures markers reside inside the parcel.
17	Debounced auto-save	Auto-save pulsing	isAutoSaving = true.	Debounce timer	{ geojson: "{...}" ,	Server updates	On failure: shows

Step #	User Action / Stage	UI / Component View	React State & Hook Updates	Events / Protocols	Data Structure / Payload	Persistence & Backend Operations	Error Handling & Guardrails
	triggers.	indicator in UI.		completes; Axios PUT dispatches.	nondevelopment areas: "[...]", trees: "[...]" }.	project record in database.	toast alert and keeps payload queued for retry.
18	User opens and completes regulatory form.	GetRegulatoryForm modal.	roadWidth = 18.0, frontSetback = 4.5.	Form submit -> Axios POST /api/projects/proj_99/regulatory.	Serialized numeric constraints object.	Regulatory parameters saved to database.	Rejects negative numbers or road widths below minimums.
19	User completes business constraints form.	BusinessConstraint[splan_27](start_span)[spal', parking n_27](end_span)s modal.	productMix = ['Residential', 'Podium'.	Form submit -> Axios POST /api/projects/proj_99/business.	JSON payload with commercial targets.	Business constraints saved to database.	Prevents submission with empty product selections.
20	User reviews consolidated summary.	InputSummary modal.	isConfirming = true.	Reads all aggregated site parameters into review table.	Consolidated project record.	None (read-only view).	Highlights any missing or conflicting parameters.
21	User clicks "Save and Continue".	Loading transition overlay.	Component prepares to unmount.	Programmatic navigation: navigate('/site-constraints').	Route state passes confirmed project context.	Flags project as ready for layout generation in database.	Prevents navigation if spatial auto-save is still pending.

Part 15: Architectural Code Reconstruction

The following modules represent an architectural reconstruction of the core Planwise frontend system based on documented behavior, standard React patterns, and stated interface contracts.

1. Dynamic Mapbox Canvas and Drawing Controller

```
// Representative reconstruction based on documented behavior
import React, { useEffect, useRef, useState, useCallback } from
'react';
import mapboxgl from 'mapbox-gl';
```

```

import MapboxDraw from '@mapbox/mapbox-gl-draw';
import axios from 'axios';
import { toast } from 'react-toastify';
import { Toolbar } from './Toolbar';
import { InputSummary } from './InputSummary';
import { validatePolygonClosure } from '../utils/geoValidation';

mapboxgl.accessToken = process.env.REACT_APP_MAPBOX_ACCESS_TOKEN;

export function MapPage({ projectId }) {
  const mapContainerRef = useRef(null);
  const mapRef = useRef(null);
  const drawRef = useRef(null);
  const saveTimeoutRef = useRef(null);

  const [hasBoundary, setHasBoundary] = useState(false);
  const [activeTool, setActiveTool] = useState(null);
  const [areaMetrics, setAreaMetrics] = useState({ grossSqm: 0,
netSqm: 0 });
  const [isSummaryOpen, setIsSummaryOpen] = useState(false);

  const persistGeospatialData = useCallback((features) => {
    if (saveTimeoutRef.current) {
      clearTimeout(saveTimeoutRef.current);
    }

    saveTimeoutRef.current = setTimeout(async () => {
      try {
        const token = localStorage.getItem('token');
        const boundaryFeature = features.find(f =>
f.properties?.entityType === 'boundary');
        const deductionFeatures = features.filter(f =>
f.properties?.entityType === 'deduction');
        const treeFeatures = features.filter(f => f.geometry.type ===
'Point');

        const payload = {
          geojson: JSON.stringify(boundaryFeature || {}),
          nondevelopableareas: JSON.stringify(deductionFeatures ||
[]),
          trees: JSON.stringify(treeFeatures || [])
        };

        await
axios.put(`http://localhost:5001/api/projects/${projectId}`, payload,
{
          headers: { Authorization: `Bearer ${token}` }
        });
      }
    }, 1000);
  });

```

```

        toast.success('Site geometries auto-saved', { autoClose: 1200
    });
    } catch (err) {
        toast.error('Auto-save failed. Geometry changes not yet
persisted.');
```

```

    }
    }, 1000);
    }, [projectId]);

const handleDrawChange = useCallback(() => {
    if (!drawRef.current) return;
    const allFeatures = drawRef.current.getAll().features;

    const boundary = allFeatures.find(f => f.properties?.entityType
=== 'boundary');
    setHasBoundary(Boolean(boundary));

    if (boundary && validatePolygonClosure(boundary)) {
        const coords = boundary.geometry.coordinates[0];
        let area = 0;
        for (let i = 0; i < coords.length - 1; i++) {
            area += (coords[i][0] * coords[i+1][1]) - (coords[i+1][0] *
coords[i][1]);
        }
        const calculatedArea = Math.abs(area * 0.5 * 111319.49 *
111319.49);
        setAreaMetrics({ grossSqm: calculatedArea, netSqm:
calculatedArea });
    } else {
        setAreaMetrics({ grossSqm: 0, netSqm: 0 });
    }

    persistGeospatialData(allFeatures);
}, [persistGeospatialData]);

useEffect(() => {
    if (mapRef.current) return;

    const map = new mapboxgl.Map({
        container: mapContainerRef.current,
        style: 'mapbox://styles/mapbox/satellite-streets-v12',
        center: [72.8777, 19.0760],
        zoom: 14,
        pitch: 0
    });

    const draw = new MapboxDraw({
        displayControlsDefault: false,

```



```

    userProperties: true
  });

  map.addControl(draw, 'top-left');
  mapRef.current = map;
  drawRef.current = draw;

  map.on('load', () => {
    map.on('draw.create', handleDrawChange);
    map.on('draw.update', handleDrawChange);
    map.on('draw.delete', handleDrawChange);
  });

  return () => {
    if (mapRef.current) {
      mapRef.current.remove();
      mapRef.current = null;
    }
  };
}, [handleDrawChange]);

const handleSelectTool = (toolKey) => {
  setActiveTool(toolKey);
  if (!drawRef.current) return;

  if (toolKey === 'boundary') {
    drawRef.current.changeMode('draw_polygon');
  } else if (toolKey === 'tree') {
    drawRef.current.changeMode('draw_point');
  } else if (toolKey === 'select') {
    drawRef.current.changeMode('simple_select');
  }
};

return (
  <div className="map-page-layout">
    <div className="hud-metrics-panel">
      <div>Gross Plot Area: {areaMetrics.grossSqm.toFixed(2)}
m²</div>
      <div>Net Developable: {areaMetrics.netSqm.toFixed(2)} m²</div>
    </div>

    <Toolbar
      hasBoundary={hasBoundary}
      activeTool={activeTool}
      onSelectTool={handleSelectTool}
      onOpenSummary={() => setIsSummaryOpen(true)}
    />
  </div>

```

```

    <div ref={mapContainerRef} className="map-canvas-container" />

    {isSummaryOpen && (
      <InputSummary
        projectId={projectId}
        metrics={areaMetrics}
        onClose={() => setIsSummaryOpen(false)}
      />
    )}
  </div>
);
}

```

2. Consolidated Input Summary and Navigation Gatekeeper

```

// Representative reconstruction based on documented behavior
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import axios from 'axios';
import { toast } from 'react-toastify';

export function InputSummary({ projectId, metrics, onClose }) {
  const navigate = useNavigate();
  const [isFinalizing, setIsFinalizing] = useState(false);

  const handleSaveAndContinue = async () => {
    setIsFinalizing(true);
    try {
      const token = localStorage.getItem('token');
      await axios.post(
        `http://localhost:5001/api/projects/${projectId}/finalize-inputs`,
        {},
        { headers: { Authorization: `Bearer ${token}` } }
      );

      toast.success('Site inputs locked. Proceeding to constraints analysis.');
```

analysis.');

```

      navigate('/site-constraints', { state: { projectId } });
    } catch (err) {
      toast.error('Failed to lock project inputs. Please retry.');
```

setIsFinalizing(false);

```

    }
  };

  return (

```

```

<div className="modal-backdrop">
  <div className="summary-dialog-card">
    <h2>Feasibility Input Summary</h2>
    <table className="summary-data-table">
      <tbody>
        <tr>
          <td>Project ID</td>
          <td>{projectId}</td>
        </tr>
        <tr>
          <td>Gross Site Area</td>
          <td>{metrics.grossSqm.toFixed(2)} m²</td>
        </tr>
        <tr>
          <td>Net Developable Area</td>
          <td>{metrics.netSqm.toFixed(2)} m²</td>
        </tr>
        <tr>
          <td>Status</td>
          <td><span className="badge badge-success">Boundary
Validated</span></td>
        </tr>
      </tbody>
    </table>

    <div className="modal-action-row">
      <button className="btn-secondary" onClick={onClose}
disabled={isFinalizing}>
        Back to Map
      </button>
      <button className="btn-primary"
onClick={handleSaveAndContinue} disabled={isFinalizing}>
        {isFinalizing ? 'Locking Inputs...' : 'Save and Continue'}
      </button>
    </div>
  </div>
</div>
);
}

```

Part 16: Production Debugging Masterclass

Systematic Debugging Scenarios

1. Authentication Fails / Returns 401 Unauthorized

- Root Causes: Token missing from localStorage, token expired past exp claim timestamp, or the Axios interceptor failed to prepend the literal prefix Bearer .
- Diagnostic Workflow:
 1. Open Chrome DevTools -> Application Tab -> Storage -> Local Storage -> Verify key 'token' exists.
 2. Copy raw token string and paste into jwt.io to inspect signature validity and timestamp expiration.
 3. Inspect Network Tab -> Select failing API request -> Headers -> Request Headers -> Verify Authorization key presence and format: Bearer eyJ....
 4. Verify CORS configuration on backend service running on port 5001 allows incoming Authorization headers.

2. Project Creation Fails / Request Drops Silently

- Root Causes: Unhandled promise rejections inside handleSubmit, payload schema mismatch (e.g., passing raw JSON objects instead of stringified fields), or missing required fields (projectTitle, landOwnership).
- Diagnostic Workflow:
 1. Inspect Network Tab -> Search for POST /api/projects -> View Request Payload.
 2. Verify payload fields match the documented contract exactly (project_title, about, land_ownership, geojson, nondevelopableareas, trees).
 3. Check console for unhandled errors: Uncaught (in promise) TypeError: Cannot read properties of undefined.
 4. Verify local backend process is actively listening on port 5001 (lsof -i :5001 or netstat -ano | findstr 5001).

3. Mapbox Canvas Does Not Render / Collapses to Zero Height

- Root Causes: Parent container lacks an explicit CSS height (common in flex layouts), WebGL hardware acceleration is disabled in the browser, or Mapbox access token is invalid/restricted.
- Diagnostic Workflow:
 1. Inspect DOM node <div class="mapbox-gl-canvas-container"> in Elements tab. Check Computed Styles -> verify height is greater than 0px. If 0px, set height: 100vh or apply flex: 1 with min-height: 0.
 2. Check Console for WebGL errors: Error: WebGL not supported. Verify via chrome://gpu that WebGL hardware acceleration is enabled.
 3. Validate access token: Verify console network calls to api.mapbox.com return HTTP 200 rather than HTTP 401 Unauthorized or HTTP 403 Forbidden.

4. Drawn Polygons Fail to Persist to Backend

- Root Causes: Event listeners for draw.create/draw.update not binding correctly, debounced timer clearing prematurely, or unhandled errors during GeoJSON stringification.
- Diagnostic Workflow:
 1. Add temporary console logging inside the Mapbox Draw event handler: map.on('draw.create', (e) => console.log('Draw Created:', e.features)).

2. If the event logs successfully, check the network tab for a subsequent PUT/POST call after the 1000ms debounce interval.
3. If no network request appears, inspect the auto-save hook to verify the timer reference (`saveTimeoutRef.current`) is not being cleared by unintended component re-renders.

5. Tree Placement Tool Remains Disabled

- Root Causes: `hasBoundary` state flag is false because the primary boundary polygon was not marked with the expected entity type property, or the state machine guard logic is evaluating to false.
- Diagnostic Workflow:
 1. In React DevTools, select the Toolbar component and inspect its incoming props: verify the value of `hasBoundary`.
 2. If `hasBoundary` is false despite a shape being visible on canvas, inspect `draw.getAll().features` in the console to confirm that the feature has valid coordinates and is recognized as a complete polygon.
 3. Verify that the toolbar's `toolActivationRules.canPlaceTrees` evaluates to true when a boundary exists.

Part 17: Performance Optimization and Enterprise Hardening

Mapbox GL JS consumes dedicated GPU memory contexts. Standard browsers limit concurrent WebGL contexts to between 8 and 16. In single-page applications with client-side routing, navigating between `/home`, `/map`, and `/site-constraints` without unmounting the map properly causes rapid context leaks. When the limit is reached, the browser forcefully drops the oldest context, crashing other canvas instances.

```
// Enforcing deterministic WebGL teardown
useEffect(() => {
  const map = new mapboxgl.Map({ /* configuration */ });
  mapRef.current = map;

  return () => {
    map.remove();
    mapRef.current = null;
  };
}, []);
```

Passing raw, high-density vector geometries across HTTP APIs degrades performance and inflates database storage. Raw browser mouse interactions emit floating-point coordinates with up to 14 decimal places. At the equator, 6 decimal places provides approximately 0.11-meter resolution, which is more than sufficient for architectural boundaries. Truncating coordinates to 6 decimal places reduces GeoJSON payload sizes by 40% to 60%. Furthermore, when users import large survey parcel files containing thousands of vertices, running client-side simplification via the Douglas-Peucker algorithm removes redundant collinear vertices without visibly altering the boundary envelope.

Technical Concern	Documented Implementation	Recommended Enterprise Implementation
Authentication Storage	Browser localStorage (vulnerable to XSS extraction).	Dual-token pattern: in-memory access token + secure, HttpOnly, SameSite=Strict refresh cookie.
State Synchronization	Local React component state with manual props drilling.	Centralized caching and state synchronization using TanStack Query (React Query) and Zustand.
GeoJSON Serialization	Stringified JSON strings passed in API payloads ("geojson": "{...}").	Native JSONB payloads on the backend; validation against strict JSON Schema / RFC 7946 specifications.
Validation Architecture	Client-side form checks combined with basic backend constraints.	Unified schema validation using Zod on the frontend and backend, with server-side computational geometry checks.
Bundle Architecture	Monolithic client bundle; synchronous library imports.	Dynamic code-splitting via React.lazy() for heavy spatial libraries (mapbox-gl, @mapbox/mapbox-gl-draw).
Rendering Strategy	Direct canvas interaction; un-memoized UI panels.	React component memoization (React.memo, useMemo) to decouple canvas operations from HUD metrics re-renders.

Part 18: Developer Contribution and Attribution Analysis

To support technical defense in an engineering review or viva examination, the developer's contributions are categorized across four distinct tiers based on the source report.

A. Clearly Documented Personal Contributions

The developer personally implemented and validated the following modules:

1. User Authentication Interface (Login): Form state management, real-time email and password validation, password visibility toggling, loading indicators, and JWT token persistence in localStorage.
2. Interactive Project Dashboard (Projects): Paginated grid of project tiles, client-side progressive "Load More" pagination (initial 7 cards, incrementing by 4), search filtering, project deletion flows, and toast feedback integration.
3. Project Creation Interface (NewProject): Controlled form inputs (projectTitle, projectDescription, landOwnership), dynamic population of the land ownership dropdown with 5 statutory options, client-side pre-flight validation, and asynchronous POST integration to `http://localhost:5001/api/projects`.

4. Interactive Spatial Workspace (MapPage): Mapbox GL JS map canvas initialization, Mapbox Draw integration, event bindings (draw.create, draw.delete), geometry validation, location search restricted to India via Mapbox Geocoder, custom viewing controls (zoom, rotate, 3D pitch toggle, grid overlay), and fixed HUD metrics display.
5. Dynamic Toolbar (Toolbar): Dual-sided floating layout with SVG icon buttons, modal toggles, image upload modal with file validation, and implementation of toolActivationRules (enforcing boundary creation before tree/deduction tools activate).
6. Constraint and Review Overlays: Implementation of GetRegulatoryForm, BusinessConstraints, GetBusinessConstraints, and InputSummary review interfaces, managing transitions forward to /site-constraints.
7. Debounced Auto-Save Engine: Debounce mechanisms in MapPage to batch and persist GeoJSON boundaries, tree markers, and deduction areas without overwhelming the backend.

B. Supporting and Collaborative Engineering

Work executed in close alignment with peers and mentors:

1. API Contract Definition: Collaborating with backend developers to establish JSON payload schemas, particularly the stringified geometry fields ("geojson", "trees", "nondevelopableareas").
2. UI/UX Alignment: Translating design mockups into responsive layouts using CSS Modules and SCSS, including preparing structural styles for dark mode theming.
3. Cross-Browser Testing and Debugging: Manual testing across browsers to ensure responsive styling and prevent WebGL canvas collapses.

C. System Capabilities Documented but Not Personally Authored

Features present in the broader Archonet platform that the developer interacted with via interfaces, but did not personally build from scratch:

1. Generative Layout Algorithms: Downstream computational geometry engines that synthesize residential unit layouts and corridor placements along net buildable envelopes.
2. Building Massing Engine: Algorithmic extrusion of 2D floor plates into 3D structural volumes based on FSI utilization rules.
3. Mapbox Core Libraries: Proprietary internal engines of @mapbox/mapbox-gl and @mapbox/mapbox-gl-draw (integrated as external third-party dependencies).

D. Unknown System Architecture Aspects

Technical implementations not detailed in the internship report:

1. Backend Service Architecture: The underlying framework (Node.js/Express, Python/FastAPI, or Go) driving the API on port 5001 is not specified.
2. Database Technology: The underlying data store (PostgreSQL/PostGIS, MongoDB, MySQL, or a BaaS like Supabase) is not documented.
3. Authentication Token Signing: The private key management and token signature verification algorithms used on the backend are not documented.

Part 19: Comprehensive Viva and Technical Interview

Guide

This section compiles 100 technical interview questions across 12 levels of proficiency, designed to prepare the developer to explain, defend, and demonstrate every facet of the Planwise system.

Level 1: Basic Web Concepts (Questions 1–8)

1. What is a Single Page Application (SPA)?

- Short Answer: A web application that loads a single HTML page and dynamically updates content via JavaScript DOM manipulation without full-page reloads.
- Deep Answer: SPAs intercept internal link clicks to prevent the browser from issuing standard HTTP GET requests for new HTML pages. Instead, client-side routers update the URL and render appropriate component trees dynamically. This architecture provides smooth, desktop-like user experiences and preserves complex client states (such as an active WebGL canvas).
- Planwise Example: Navigating between /home, /project-selection, and /map occurs without full-page reloads, preventing the Mapbox canvas from destroying its WebGL context.
- Common Wrong Answer: Claiming that SPAs make zero network calls. SPAs make frequent background HTTP requests for JSON data while keeping the HTML shell intact.

2. What is the Document Object Model (DOM)?

- Short Answer: A programming interface representing an HTML document as a tree structure of objects that can be manipulated via JavaScript.
- Deep Answer: The browser parses raw HTML strings into a tree of C++ nodes in memory. JavaScript running in the browser interacts with this tree via the DOM API (document.createElement, addEventListener). Because directly mutating the browser DOM can be computationally expensive, modern libraries use virtual representations to batch updates.
- Planwise Example: React manages the DOM nodes for forms and toolbars, while Mapbox GL JS directly manipulates its own dedicated <canvas> DOM element.
- Common Wrong Answer: Confusing the browser's native DOM with React's in-memory Virtual DOM.

3. What is the difference between client-side rendering (CSR) and server-side rendering (SSR)?

- Short Answer: CSR renders HTML in the browser via JavaScript; SSR compiles HTML strings on the server before sending them to the client.
- Deep Answer: In CSR, the browser downloads a minimal HTML shell alongside a bundled JavaScript payload. The browser executes the JavaScript to build the DOM, fetch data, and render the UI. In SSR, the server renders the initial HTML on each request, delivering fully formed markup that displays before JavaScript executes. CSR is well-suited for private, highly interactive dashboards, whereas SSR is ideal for public, SEO-dependent

websites.

- Planwise Example: Planwise uses client-side rendering because it is an authenticated dashboard with complex WebGL interactions that cannot be pre-rendered on a server.
- Common Wrong Answer: Claiming SSR is universally superior to CSR; SSR adds server compute overhead and cannot server-render interactive WebGL canvases.

4. What is the browser event loop?

- Short Answer: The concurrency runtime mechanism that coordinates the Call Stack, Microtask Queue, and Macrotask Queue to execute non-blocking asynchronous JavaScript.
- Deep Answer: JavaScript runs on a single thread. Synchronous operations execute immediately on the Call Stack. Asynchronous callbacks are offloaded to browser Web APIs. When asynchronous operations resolve, their callbacks enter either the Microtask Queue (Promises, queueMicrotask) or the Macrotask Queue (setTimeout, user UI events). Once the Call Stack is empty, the Event Loop prioritizes the Microtask Queue, processing all pending microtasks before handling the next macrotask.
- Planwise Example: Auto-save debounce timers run as macrotasks via setTimeout, whereas Axios network resolution promises execute as high-priority microtasks.
- Common Wrong Answer: Believing that setTimeout(fn, 1000) guarantees execution at exactly 1000ms; it only guarantees execution no earlier than 1000ms, depending on call stack availability.

5. What are CSS Modules and why are they used?

- Short Answer: A CSS file format where all class names are scoped locally to the importing component by default.
- Deep Answer: Standard CSS class names live in a single global namespace, creating risks of unintended style collisions in large applications. CSS Modules solve this by compiling human-readable class names into unique, hashed strings at build time. This ensures style isolation for individual components.
- Planwise Example: Used to style Toolbar and Projects components so that identical class names like .error-field do not conflict across different forms.
- Common Wrong Answer: Attempting to target CSS Module classes from external stylesheets using raw, unhashed strings.

6. What is the purpose of the HTTP Authorization header?

- Short Answer: A standard request header used to provide authentication credentials verifying the client's identity to a server.
- Deep Answer: Under RFC 7235, the Authorization header transmits credentials containing a specific authentication scheme followed by parameters. In token-based architectures, applications use the Bearer scheme (Authorization: Bearer <token>). The server inspects this header before routing requests to protected controllers, returning HTTP 401 Unauthorized if the token is missing, malformed, or expired.
- Planwise Example: Every Axios request fetching projects or saving geometries attaches Bearer <token> retrieved from localStorage.
- Common Wrong Answer: Omitting the literal prefix Bearer before the token string, causing

backend parsers to reject the request.

7. What is JSON and how does serialization work?

- Short Answer: JavaScript Object Notation—a text format for structured data; serialization converts in-memory objects into text strings.
- Deep Answer: In-memory JavaScript objects cannot travel directly across network sockets. `JSON.stringify()` serializes JavaScript objects into UTF-8 text strings, stripping out functions, undefined values, and cyclic references. The receiving service parses the string back into native data structures using `JSON.parse()`.
- Planwise Example: Geometry collections generated by Mapbox Draw are serialized via `JSON.stringify(draw.getAll())` before transmission in the "geojson" API payload field.
- Common Wrong Answer: Attempting to stringify objects with circular references, which throws an unhandled `TypeError`: Converting circular structure to JSON.

8. What is the difference between `localStorage` and `sessionStorage`?

- Short Answer: `localStorage` persists data permanently across browser restarts; `sessionStorage` clears data when the specific browser tab closes.
- Deep Answer: Both belong to the browser Web Storage API, providing synchronous key-value storage up to roughly 5MB per origin. `localStorage` is shared across all tabs and windows with the same origin and persists until explicitly cleared. `sessionStorage` is scoped to a single tab's browsing context and is wiped as soon as that tab is closed.
- Planwise Example: Planwise stores the authentication JWT in `localStorage` so users stay logged in across multiple tabs and browser sessions.
- Common Wrong Answer: Assuming `sessionStorage` is shared between two tabs open to the same URL; it is isolated to each tab.

Level 2: React Core Fundamentals (Questions 9–16)

9. What is the Virtual DOM and how does reconciliation work?

- Short Answer: A lightweight JavaScript representation of the real DOM; reconciliation compares virtual trees to apply minimal DOM updates.
- Deep Answer: Directly modifying the browser DOM is expensive because changes can trigger layout reflows and repaints. React maintains two virtual DOM trees in memory. When state changes, a new virtual tree is constructed. React runs its diffing algorithm (Fiber reconciler) to compare the new tree against the old one. It calculates the minimum set of DOM mutations needed and applies them in a single batch.
- Planwise Example: When `areaMetrics` updates in `MapPage`, React updates only the specific text nodes in the HUD metrics panel without re-rendering the Mapbox canvas.
- Common Wrong Answer: Believing that the Virtual DOM is inherently faster than direct DOM manipulation; its purpose is predictable, declarative state management with consistent performance.

10. What are the core rules of React Hooks?

- Short Answer: Call hooks only at the top level of functional components, and call hooks

only from React functions.

- Deep Answer: React tracks hook state internally using ordered arrays indexed by execution sequence. If hooks are placed inside loops, conditionals, or nested functions, the execution order changes between renders. This shifts the internal index, associating state variables with the wrong hooks and causing unpredictable runtime bugs.
- Planwise Example: Placing `useState` and `useEffect` calls strictly at the top level of `MapPage` before any conditional returns or modal checks.
- Common Wrong Answer: Wrapping a `useEffect` inside an `if (hasBoundary)` condition rather than placing the condition inside the effect callback.

11. What is the difference between `useState` and `useRef`?

- Short Answer: Updating `useState` triggers a component re-render; updating `useRef` mutates a value silently without re-rendering.
- Deep Answer: `useState` is intended for values that directly drive visual JSX elements. Its updater function notifies React to schedule a re-render. `useRef` returns a stable object with a mutable `.current` property that persists across renders. Changing `.current` produces no notifications and schedules no renders, making it ideal for storing non-visual references or third-party instances.
- Planwise Example: The Mapbox Map instance is stored in `useRef` to prevent re-rendering the canvas, while modal visibility flags are stored in `useState`.
- Common Wrong Answer: Mutating a `useRef.current` value and expecting the visual UI to update automatically.

12. How does the dependency array in `useEffect` control execution?

- Short Answer: It tells React to re-run the effect only when specified values change between renders; empty array runs once on mount.
- Deep Answer: After React completes the DOM commit phase, it evaluates the values in the effect's dependency array using `Object.is()` shallow comparisons. If every value matches its previous render value, the effect is skipped. If an array is omitted entirely, the effect executes after every single render. If an empty array (`[]`) is passed, it runs once after initial mount and cleans up on unmount.
- Planwise Example: Fetching the project list inside `Projects` runs inside a `useEffect` with an empty dependency array (`[]`) to load data once on mount.
- Common Wrong Answer: Omitting dependencies referenced inside the effect closure, leading to stale closures that read outdated state variables.

13. What is a "stale closure" in a React hook?

- Short Answer: An asynchronous callback or effect capturing an outdated state variable from a previous render cycle.
- Deep Answer: JavaScript functions retain access to variables declared in their outer lexical scope (closures). When React re-renders a functional component, a fresh closure is created with updated state values. If a long-running timer, promise, or event listener retains a reference to an earlier closure, it reads the variable values as they existed when that earlier closure was created, rather than the latest state.
- Planwise Example: If an auto-save timer captures `hasBoundary` without declaring it in its

dependencies, it might read `hasBoundary` as `false` even after a boundary has been drawn.

- Common Wrong Answer: Attempting to fix state closures by removing dependencies rather than using functional state updaters (e.g., `setCount(prev => prev + 1)`).

14. What are controlled components versus uncontrolled components?

- Short Answer: Controlled component values are driven by React state; uncontrolled component values are handled directly by the native DOM.
- Deep Answer: In a controlled input, React serves as the single source of truth. The input accepts a value prop bound to state and an `onChange` prop that updates that state on every keystroke. In an uncontrolled input, the input maintains its internal DOM state, and React reads its value on demand using a `useRef` to query the DOM node directly.
- Planwise Example: The `projectTitle` input and `landOwnership` dropdown in `NewProject` are controlled components.
- Common Wrong Answer: Switching an input from uncontrolled to controlled during its lifecycle by passing `undefined` as the initial value.

15. What is prop drilling and how is it resolved?

- Short Answer: Passing props down through multiple layers of intermediate components that do not need them directly.
- Deep Answer: As component trees grow, top-level state often needs to reach deeply nested children. Passing these variables down through intermediate components clutters their interfaces and couples them to data they do not use. Solutions include component composition (passing fully formed JSX via children), the React Context API, or external state stores.
- Planwise Example: Passing `projectId` from `MapPage` through to nested form and summary overlays.
- Common Wrong Answer: Introducing complex global state management libraries (e.g., Redux) to solve simple prop drilling that could be resolved with basic component composition.

16. Why are keys required when rendering dynamic lists in React?

- Short Answer: Keys give elements stable identities, allowing React's reconciler to track additions, moves, and deletions efficiently.
- Deep Answer: During reconciliation, React compares lists of children. Without keys, if an item is inserted at the beginning of an array, React mutates every existing DOM node to match the shifted index. With unique, stable keys, React compares keys across renders, leaving existing DOM nodes untouched and simply inserting the new node at the correct position.
- Planwise Example: Rendering the paginated list of project tiles in `Projects` using `project._id` as the list item key.
- Common Wrong Answer: Using array index (`key={index}`) for dynamic lists, which causes subtle UI and state bugs when items are reordered or deleted.

Level 3: Advanced JavaScript and Asynchronous Patterns (Questions

17–24)

17. What is the difference between synchronous and asynchronous code execution?

- Short Answer: Synchronous code blocks the execution thread until completion; asynchronous code offloads work and continues execution immediately.
- Deep Answer: Synchronous instructions execute sequentially on the single main JavaScript thread; long operations freeze the browser interface. Asynchronous operations (such as HTTP calls or timers) are offloaded to browser background threads. When finished, their callbacks are placed on the task queues to run when the call stack clears, keeping the interface responsive.
- Planwise Example: Axios network calls to fetch projects run asynchronously, keeping the UI interactive while waiting for server responses.
- Common Wrong Answer: Writing computationally heavy geometric loops directly on the main thread, causing frame drops during map interactions.

18. How do JavaScript Promises work under the hood?

- Short Answer: A Promise is an object representing the eventual completion or failure of an asynchronous operation.
- Deep Answer: A Promise exists in one of three states: pending, fulfilled, or rejected. Once settled, its state becomes immutable. Promises provide `.then()` and `.catch()` handlers that register callbacks into the microtask queue. This guarantees that resolved callbacks execute asynchronously as soon as the current synchronous execution context completes.
- Planwise Example: Axios methods return Promises that resolve with API data or reject with server error responses.
- Common Wrong Answer: Forgetting to catch Promise rejections, which triggers `UnhandledPromiseRejection` warnings and can leave UI loading spinners spinning indefinitely.

19. What is the syntactic difference between `async/await` and raw Promises?

- Short Answer: `async/await` is syntactic sugar over Promises, allowing asynchronous code to be written with cleaner, synchronous-looking syntax.
- Deep Answer: Declaring a function `async` guarantees it returns a Promise. The `await` keyword pauses the function's execution until the awaited Promise settles, unwrapping its resolved value. Under the hood, the JavaScript engine suspends the function context, offloads the continuation to the microtask queue, and frees the main thread to handle other tasks.
- Planwise Example: The `handleSubmit` function in `NewProject` uses `await axios.post(...)` wrapped in a `try/catch` block for clean error handling.
- Common Wrong Answer: Forgetting that `await` pauses execution only within the `async` function; outer synchronous code continues executing immediately.

20. How does the AbortController API cancel asynchronous requests?

- Short Answer: It provides an abort signal object that can communicate with asynchronous operations and cancel them mid-flight.
- Deep Answer: An AbortController instance exposes a `.signal` property and an `.abort()` method. Passing this signal to a fetch or Axios request links the underlying network socket to the controller. Invoking `controller.abort()` signals the browser to terminate the network connection immediately, rejecting the pending Promise with an `AbortError`.
- Planwise Example: Cancelling a pending auto-save request when a newer geometry change occurs, ensuring obsolete coordinates are not written over newer ones.
- Common Wrong Answer: Treating an aborted request error like a normal API failure, which can trigger confusing error alerts for operations that were intentionally cancelled.

21. What is closure and how does lexical scoping work in JavaScript?

- Short Answer: A closure is a function bundled together with references to its surrounding lexical state.
- Deep Answer: In JavaScript, variable access is determined by where functions are declared in the source code (lexical scoping). An inner function retains access to variables declared in its enclosing parent scopes even after the parent function has finished executing and returned. This mechanism enables private variables, functional factories, and curried operations.
- Planwise Example: Debounce functions rely on closures to retain access to the active `timeoutId` across multiple function calls.
- Common Wrong Answer: Creating unintended closures inside loops that capture shared mutable variables, leading to unexpected values when callbacks run.

22. What is the difference between shallow equality and deep equality?

- Short Answer: Shallow equality compares memory references; deep equality compares every nested value within objects or arrays.
- Deep Answer: Shallow comparison (`===` or `Object.is()`) checks whether two operands share the same primitive value or point to the exact same reference in memory. Two distinct object literals containing identical keys and values evaluate to false under shallow comparison. Deep equality recursively inspects every nested key and value to verify structural equivalence.
- Planwise Example: React uses shallow equality to check hook dependencies; passing a newly constructed object literal every render always triggers the hook, even if the values inside did not change.
- Common Wrong Answer: Mutating an existing array in place (e.g., `list.push(item)`) and passing it to state; because the reference did not change, React skips re-rendering.

23. What are JavaScript modules (ESM) versus CommonJS?

- Short Answer: ESM uses static `import/export` syntax; CommonJS uses dynamic `require()` and `module.exports`.
- Deep Answer: ECMAScript Modules (ESM) are parsed statically by modern runtimes and

bundlers, allowing unused code to be identified and eliminated at build time (tree-shaking). CommonJS resolves module dependencies dynamically at runtime, which makes tree-shaking significantly harder. Modern frontend stacks use ESM natively.

- Planwise Example: Importing modules using static imports: `import mapboxgl from 'mapbox-gl'`;
- Common Wrong Answer: Mixing `import` and `module.exports` statements in the same file, which can cause module bundling and build errors.

24. How does garbage collection work in modern JavaScript runtimes?

- Short Answer: The engine automatically frees memory when objects are no longer reachable from the root execution context.
- Deep Answer: Engines (such as V8) employ generational, mark-and-sweep garbage collection. The collector begins at root references (the global window object, active call stack frames, DOM nodes) and traverses all outgoing references. Any object in memory that cannot be reached through this traversal is marked as unreachable and reclaimed. Retaining dangling event listeners or global variables prevents memory from being freed, causing memory leaks.
- Planwise Example: Removing Mapbox canvas event listeners on component unmount ensures the WebGL canvas and its memory can be garbage collected.
- Common Wrong Answer: Retaining global references to large GeoJSON structures, preventing the garbage collector from reclaiming the memory.

Level 4: Network Protocols and REST APIs (Questions 25–32)

25. What are the key architectural constraints of a RESTful API?

- Short Answer: Stateless communication, client-server separation, cacheable responses, uniform interfaces, and layered system design.
- Deep Answer: REST (Representational State Transfer) treats server operations as standard interactions with identifiable resources via URIs. The server does not store client session state; every request must contain all the information needed to process it. Standard HTTP verbs define operations: GET reads, POST creates, PUT replaces, PATCH modifies, and DELETE removes.
- Planwise Example: Project operations follow RESTful conventions: GET `/api/projects` lists projects, and POST `/api/projects` creates one.
- Common Wrong Answer: Using POST requests for every operation instead of following semantic REST verbs.

26. What is Cross-Origin Resource Sharing (CORS)?

- Short Answer: A browser security mechanism that restricts web applications from requesting resources on a different domain, protocol, or port.
- Deep Answer: Under the Same-Origin Policy, browsers prevent client-side scripts from reading cross-origin HTTP responses. When a frontend at `http://localhost:3000` calls an API at `http://localhost:5001`, the browser issues an automated OPTIONS preflight request. The server must respond with explicit CORS headers permitting the client to make the call.

- Planwise Example: The backend running on port 5001 must return CORS headers allowing requests from the React frontend running on port 3000.
- Common Wrong Answer: Believing that CORS is a server-side security barrier; CORS is enforced by the browser to protect users from malicious cross-origin scripts.

27. What is an HTTP preflight request?

- Short Answer: An automatic OPTIONS request sent by the browser to verify that the server permits a proposed cross-origin call.
- Deep Answer: If a cross-origin HTTP call uses methods other than GET/POST or contains custom headers (such as Authorization: Bearer <token>), it is considered a non-simple request. Before sending the actual request, the browser sends an OPTIONS request asking the server for permission. If the server responds with matching Access-Control-Allow-* headers, the browser sends the primary request.
- Planwise Example: Adding the Authorization header to API calls triggers a preflight OPTIONS request that port 5001 must accept.
- Common Wrong Answer: Failing to handle OPTIONS requests on backend routers, which causes all authenticated frontend requests to fail CORS checks.

28. What do HTTP status codes 200, 201, 400, 401, 403, and 500 indicate?

- Short Answer: Standard response codes indicating success (2xx), client errors (4xx), or server failures (5xx).
- Deep Answer:
 - 200 OK: Standard success response for reads or updates.
 - 201 Created: The request succeeded and a new resource was created.
 - 400 Bad Request: The server cannot process the request due to malformed syntax or invalid parameters.
 - 401 Unauthorized: Authentication is required and has failed or was not provided.
 - 403 Forbidden: The client is authenticated but lacks permission to access the resource.
 - 500 Internal Server Error: An unexpected server failure occurred.
- Planwise Example: Successful project creation returns 201 Created, while an invalid token returns 401 Unauthorized.
- Common Wrong Answer: Returning 200 OK with an error message inside the JSON body, which tricks client libraries into treating failures as successes.

29. What is the difference between Axios and the native Fetch API?

- Short Answer: Axios handles JSON transformation automatically and rejects on error status codes; Fetch requires manual parsing and rejects only on network failures.
- Deep Answer: Axios transforms request and response data to and from JSON automatically, supports request and response interceptors, supports upload progress tracking, and rejects its Promise whenever the server responds with a 4xx or 5xx status code. Native window.fetch resolves successfully on 4xx/5xx responses and requires manual .json() body stream parsing.
- Planwise Example: Planwise uses Axios for API requests to take advantage of automatic error handling and simplified token header injection.

- Common Wrong Answer: Assuming fetch will throw an error on an HTTP 500 response; fetch rejects only if the network connection fails entirely.

30. How do Axios interceptors work?

- Short Answer: Middleware functions that run globally before a request is sent or before a response is passed to component code.
- Deep Answer: Interceptors allow applications to centralize cross-cutting network concerns. A request interceptor can read tokens from storage and inject Authorization headers on every outgoing call automatically. A response interceptor can inspect incoming responses, capture 401 Unauthorized codes, clear local storage, and redirect users to /login without duplicating logic across components.
- Planwise Example: Using a response interceptor to catch expired JWTs and redirect unauthenticated users to /login.
- Common Wrong Answer: Forgetting to return the config object in a request interceptor, which halts the outgoing request.

31. What is an idempotent HTTP method?

- Short Answer: An HTTP method that produces the exact same server state regardless of how many times the identical request is executed.
- Deep Answer: Under RFC 7231, GET, PUT, DELETE, HEAD, and OPTIONS are idempotent. Sending a PUT request with an updated geometry payload 10 times in a row leaves the database in the exact same state as sending it once. Conversely, POST is non-idempotent; sending the same POST /api/projects payload 5 times creates 5 separate project records.
- Planwise Example: Project creation uses non-idempotent POST, while geometry auto-save uses idempotent PUT.
- Common Wrong Answer: Using POST for auto-save operations, which risks creating duplicate records if retries occur.

32. What is the difference between JSON and multipart/form-data payloads?

- Short Answer: JSON transmits structured text; multipart/form-data transmits binary files alongside structured key-value pairs.
- Deep Answer: application/json encodes structured text data as UTF-8 characters. To send raw binary files, applications use multipart/form-data. This format separates fields using distinct boundary delimiter strings, allowing raw byte streams to be transmitted efficiently without the overhead of Base64 encoding.
- Planwise Example: Project metadata is sent as application/json, while project photos are uploaded via the Toolbar modal using multipart/form-data.
- Common Wrong Answer: Manually setting the Content-Type: multipart/form-data header in Axios rather than letting the browser generate the header along with its required boundary string.

Level 5: Security, Authentication, and Session Management

(Questions 33–40)

33. What are the three parts of a JSON Web Token (JWT)?

- Short Answer: Header, Payload, and Signature, separated by period characters (.).
- Deep Answer: The Header identifies the hashing algorithm (e.g., HMAC SHA256). The Payload contains claims (user ID, expiration, roles). Both parts are Base64URL-encoded strings. The Signature is generated by taking the encoded header, the encoded payload, and a secret key, and hashing them using the specified algorithm. The server verifies this signature to confirm that the payload has not been tampered with.
- Planwise Example: The JWT returned on successful login encodes the user's session and is sent with subsequent API requests.
- Common Wrong Answer: Believing that JWT data is encrypted; Base64URL encoding is easily decoded by anyone who intercepts the token. Sensitive data should never be stored in the payload.

34. What is the difference between authentication and authorization?

- Short Answer: Authentication verifies who you are; authorization determines what you are permitted to do.
- Deep Answer: Authentication is the process of confirming identity (e.g., verifying an email and password against stored credentials and issuing a token). Authorization evaluates whether that authenticated identity has permission to perform a specific action (e.g., verifying that User A has permission to delete Project 99).
- Planwise Example: The Login component handles authentication; backend checks verifying project ownership before deletion handle authorization.
- Common Wrong Answer: Assuming that a valid JWT automatically grants access to all backend resources.

35. What is Cross-Site Scripting (XSS) and how does it threaten tokens in localStorage?

- Short Answer: An attack where malicious JavaScript is injected into a site; it can access localStorage and steal stored authentication tokens.
- Deep Answer: If an application renders untrusted user input without sanitization, an attacker can inject malicious <script> tags. Once executing in the user's browser, that script has full access to the origin's localStorage. The script can read the token via localStorage.getItem('token') and transmit it to an external server, compromising the account.
- Planwise Example: Storing the Planwise JWT in localStorage requires input validation to prevent XSS vulnerabilities.
- Common Wrong Answer: Assuming React's built-in JSX escaping completely eliminates XSS risks; unsafe third-party packages can still introduce vulnerabilities.

36. What is Cross-Site Request Forgery (CSRF)?

- Short Answer: An attack that tricks an authenticated user's browser into executing

unauthorized actions on a trusted site.

- Deep Answer: If a site relies on automatically submitted browser cookies for authentication, an attacker can host a malicious page that submits forms to the target site. The victim's browser automatically attaches the authentication cookies, and the target server executes the request as if the user intended it. Token-based authentication using Authorization: Bearer headers is inherently immune to CSRF because malicious sites cannot read or inject the custom header.
- Planwise Example: Planwise's use of explicit Authorization headers protects its API from standard CSRF attacks.
- Common Wrong Answer: Assuming CSRF protection is unnecessary when using cookies without SameSite flags.

37. What are HttpOnly and SameSite cookie flags?

- Short Answer: HttpOnly blocks JavaScript from reading the cookie; SameSite prevents the cookie from being sent with cross-site requests.
- Deep Answer: The HttpOnly flag instructs the browser that the cookie must never be accessible via JavaScript DOM APIs (document.cookie), neutralizing token theft via XSS. The SameSite flag (Strict or Lax) instructs the browser not to attach the cookie to cross-site requests originating from external origins, preventing CSRF attacks.
- Planwise Example: Upgrading Planwise from localStorage to HttpOnly cookies provides enterprise-grade token security.
- Common Wrong Answer: Setting SameSite=None without also enabling the Secure flag, which causes modern browsers to reject the cookie.

38. How does a client handle JWT expiration?

- Short Answer: The server rejects expired tokens with HTTP 401, and the client clears the expired token and redirects to login.
- Deep Answer: JWT payloads include an exp (expiration) timestamp. When this timestamp passes, the server rejects requests with HTTP 401 Unauthorized. A client-side Axios response interceptor catches this 401 status, clears localStorage.removeItem('token'), updates authentication state, and redirects the user to /login with an informative message.
- Planwise Example: Clearing expired sessions when navigating protected routes or making API calls.
- Common Wrong Answer: Attempting to parse the token on the client to check expiration without catching server-side 401 responses, which misses cases where a token was revoked early.

39. What is a Refresh Token rotation strategy?

- Short Answer: Using short-lived access tokens alongside single-use refresh tokens to maintain secure, continuous user sessions.
- Deep Answer: The client receives a short-lived access token (valid for 5–15 minutes) and a long-lived refresh token (valid for 7 days) stored in an HttpOnly cookie. When the access token expires, the client calls a refresh endpoint. The server validates the refresh token, revokes it, and issues a new access token and a new refresh token. If a revoked refresh token is ever reused, the server invalidates all related sessions, protecting against

stolen tokens.

- Planwise Example: A production upgrade path for Planwise to eliminate localStorage token exposure while keeping user sessions active during long planning workflows.
- Common Wrong Answer: Allowing refresh tokens to be used multiple times without rotation, which undermines their security value.

40. Why must passwords never be stored in plain text?

- Short Answer: Storing plain text passwords exposes user credentials directly if the database is breached.
- Deep Answer: Passwords must be hashed using slow, computationally intensive, salted cryptographic algorithms (such as bcrypt, Argon2, or PBKDF2). A salt (a unique, random string) is added to each password before hashing to defeat precomputed rainbow table attacks. The computational cost factor can be increased over time as hardware speeds improve, protecting against brute-force cracking.
- Planwise Example: The backend validating credentials submitted by the Login form verifies hashes rather than comparing plain text.
- Common Wrong Answer: Using fast hashing algorithms like MD5 or SHA-256 for passwords; these are designed for speed and can be brute-forced in seconds.

Level 6: Geospatial Engineering and Mapbox GL JS (Questions 41–50)

41. How does Mapbox GL JS render maps in the browser?

- Short Answer: It uses client-side WebGL to render vector tile data on the GPU at 60 frames per second.
- Deep Answer: Traditional mapping libraries load static pre-rendered raster image tiles (.png). Mapbox GL JS downloads mathematical vector data (points, lines, polygons) packaged in protocol buffers. The library executes shaders on the client's GPU via WebGL to draw, color, and style these geometries in real time according to a style JSON specification. This enables dynamic styling, smooth zooming, rotation, and 3D tilting.
- Planwise Example: Renders parcel boundaries smoothly at variable angles and zoom levels on MapPage.
- Common Wrong Answer: Confusing vector tiles with raster tiles; vector tiles are mathematical coordinate definitions, not static images.

42. What is WebGL and what causes a CONTEXT_LOST_WEBGL error?

- Short Answer: A JavaScript API for rendering hardware-accelerated 3D graphics inside a canvas element without plugins.
- Deep Answer: WebGL connects browser JavaScript directly to the device's graphics processing unit (GPU). Browsers place strict limits on how many concurrent WebGL rendering contexts can exist simultaneously (typically 8 to 16). If an application mounts new Mapbox canvases without destroying old ones, the browser runs out of GPU memory and forcibly terminates the oldest context, throwing CONTEXT_LOST_WEBGL and crashing the canvas.
- Planwise Example: Properly calling map.remove() when unmounting MapPage avoids exhausting WebGL contexts.

- Common Wrong Answer: Hiding a map with CSS (`display: none`) instead of cleanly unmounting it, which keeps its WebGL context alive in GPU memory.

43. What is the difference between a Mapbox Source and a Mapbox Layer?

- Short Answer: A Source provides the raw geographic data; a Layer defines how that data is visually styled and rendered.
- Deep Answer: A Source holds spatial coordinates and properties (e.g., a GeoJSON FeatureCollection or vector tile URL). A Layer references that source by ID and defines its visual styling: fill color, opacity, line width, stroke dash patterns, or 3D extrusion heights. Multiple layers can reference a single source to render different visual aspects (e.g., one layer for polygon fill and another for a bold outline).
- Planwise Example: A single parcel GeoJSON source feeds one layer for the semi-transparent property fill and a second layer for the solid boundary outline.
- Common Wrong Answer: Attempting to add a Layer that references a Source ID before that Source has been registered via `map.addSource()`.

44. What are the operational modes in Mapbox Draw?

- Short Answer: Editing states that determine how pointer interactions behave on the map canvas.
- Deep Answer: `@mapbox/mapbox-gl-draw` uses modes to manage user interactions:
 - `simple_select`: Allows selecting, moving, or deleting entire shapes.
 - `direct_select`: Exposes individual vertices and midpoints on a shape for fine editing.
 - `draw_polygon`: Captures sequential mouse clicks to construct closed polygonal rings.
 - `draw_point`: Captures a single click coordinate to instantiate point markers.
- Planwise Example: The Toolbar switches modes programmatically: clicking the boundary tool activates `draw_polygon`, while clicking the tree tool activates `draw_point`.
- Common Wrong Answer: Calling `draw.changeMode()` before Mapbox Draw has finished initializing, throwing unhandled null reference errors.

45. What Mapbox events fire during polygon creation and editing?

- Short Answer: `draw.create`, `draw.update`, and `draw.delete`.
- Deep Answer: When a user finishes drawing a polygon (by double-clicking the final vertex), Mapbox Draw dispatches the `draw.create` event. If the user shifts the shape or drags any of its vertices in `direct_select` mode, it dispatches `draw.update` on every change. When an active shape is removed, it dispatches `draw.delete`. Listening to these three events ensures complete synchronization with application state.
- Planwise Example: `MapPage` listens to all three events to recalculate HUD metrics and trigger debounced auto-saves.
- Common Wrong Answer: Listening only to `draw.create`, which causes vertex adjustments made during subsequent edits to be lost.

46. How does Mapbox Geocoder work and how is it restricted to India?

- Short Answer: It converts address search queries into geographic coordinates via API,

filtered by country code.

- Deep Answer: Mapbox Geocoder sends text queries to the Mapbox Geocoding API, returning ranked geographic matches with coordinates. To restrict results to India, the configuration passes countries: 'in' and sets a bounding box parameter (bbox) matching India's geographic bounds ([68.1, 6.5, 97.4, 35.5]), filtering out irrelevant international matches.
- Planwise Example: The address search bar on MapPage uses this restriction to keep site searches focused on Indian parcels.
- Common Wrong Answer: Omitting the country filter, which causes local Indian search terms to return irrelevant matching street names in other countries.

47. How is 3D view toggling implemented in Mapbox GL JS?

- Short Answer: By programmatically animating the camera's pitch property between flat (0°) and tilted (60°) perspectives.
- Deep Answer: Mapbox GL JS represents the camera in three dimensions. Orthogonal, top-down views use a pitch of 0° . Calling `map.easeTo({ pitch: 60, bearing: -20, duration: 1000 })` smoothly animates the camera to an oblique angle, displaying vector building extrusions, terrain elevation, and massing models in 3D perspective.
- Planwise Example: The 3D View button in the Toolbar toggles pitch to allow inspection of topographic site conditions.
- Common Wrong Answer: Increasing pitch without adjusting camera bearing or zoom, which can cause nearby terrain features to clip through the camera plane.

48. What is the difference between EPSG:4326 and EPSG:3857 coordinate systems?

- Short Answer: EPSG:4326 uses spherical degrees (latitude/longitude); EPSG:3857 uses planar meters (Web Mercator projection).
- Deep Answer: EPSG:4326 (WGS84) defines coordinates as angles on an ellipsoidal model of Earth, expressing locations as [Longitude, Latitude] in degrees. It is the mandatory format for GeoJSON. EPSG:3857 (Web Mercator) projects those spherical coordinates onto a flat, planar surface measured in meters. It is used by map display engines to render vector tiles without angular distortion.
- Planwise Example: Parcel coordinates are stored and sent via API in EPSG:4326, but the Mapbox engine projects them to EPSG:3857 to paint the screen.
- Common Wrong Answer: Attempting to calculate Euclidean polygon areas using raw EPSG:4326 degree values, which yields meaningless area numbers.

49. How does Mapbox calculate distances and areas accurately on a curved surface?

- Short Answer: Using spherical geometry formulas (Haversine for distances, spherical excess for areas) or geodesic library algorithms.
- Deep Answer: Because Earth is an oblate spheroid, calculating distances and areas using simple flat-plane Cartesian math produces substantial errors that worsen at higher latitudes. Accurate measurements use geodesic mathematics: the Haversine formula calculates great-circle distances, and spherical excess algorithms compute true surface

- areas by accounting for the Earth's curvature.
- Planwise Example: The HUD metrics panel calculates parcel square footage from GeoJSON coordinates using geodesic math.
- Common Wrong Answer: Calculating polygon area with basic planar trapezoidal math, which produces inaccurate site measurements.

50. What is a coordinate reference system (CRS) transform?

- Short Answer: The mathematical conversion of coordinates from one spatial reference system to another.
- Deep Answer: Different spatial reference systems use different reference ellipsoids and projections. Converting local municipal survey coordinates into web-compatible GPS coordinates requires applying mathematical transformation matrices to re-project every coordinate pair into WGS84 degrees (EPSG:4326).
- Planwise Example: Transforming municipal revenue survey drawings into WGS84 GeoJSON features that align with satellite basemaps.
- Common Wrong Answer: Displaying untransformed CAD coordinates directly on a web map, which causes shapes to appear in the ocean at coordinates [0, 0].

Level 7: GeoJSON Specification and Spatial Modeling (Questions 51–58)

51. What is the fundamental coordinate ordering mandated by RFC 7946 GeoJSON?

- Short Answer: [Longitude, Latitude], with optional Elevation ([X, Y, Z]).
- Deep Answer: Under Section 3.1.1 of the RFC 7946 standard, coordinates must be ordered as easting then northing—corresponding to [Longitude, Latitude]. This matches standard Cartesian mathematical coordinate ordering ([X, Y]). However, popular verbal usage and some mapping APIs (like Google Maps) use [Latitude, Longitude]. Reversing this order in GeoJSON places coordinates in the wrong hemisphere.
- Planwise Example: Coordinates in Mumbai must be structured as [72.8777, 19.0760], never [19.0760, 72.8777].
- Common Wrong Answer: Inverting coordinate pairs, which places Indian land parcels in Antarctica or the Southern Ocean.

52. What defines a valid GeoJSON LinearRing in a Polygon?

- Short Answer: A closed array of four or more coordinate pairs where the first and last coordinates are identical.
- Deep Answer: An RFC 7946 linear ring must contain at least four coordinate pairs (representing a minimum of three distinct vertices plus the closing vertex). The closing coordinate must be an exact duplicate of the initial coordinate ($[X_0, Y_0] == [X_N, Y_N]$). If the closing coordinate does not match the opening coordinate, the shape is considered an open LineString rather than a valid Polygon.
- Planwise Example: Verifying that drawn boundary polygons are closed before serializing them for auto-save.

- Common Wrong Answer: Passing only three coordinate pairs for a triangle, omitting the duplicate fourth coordinate required to close the ring.

53. What is the difference between an exterior ring and an interior ring in GeoJSON?

- Short Answer: The exterior ring defines the outer boundary; interior rings define "holes" or cutouts within that boundary.
- Deep Answer: A GeoJSON Polygon's coordinates property is an array of linear rings. The first ring in the array is always the exterior ring, establishing the outer perimeter. Any subsequent linear rings in the array represent interior rings—holes cut out of the exterior polygon. Surface area within interior rings is subtracted from total polygon area calculations.
- Planwise Example: Representing non-developable zones (like public reservations) as interior rings cut out of the parcel boundary.
- Common Wrong Answer: Reversing ring order by placing an interior cutout first, which causes spatial engines to invert the shape.

54. What is a GeoJSON FeatureCollection?

- Short Answer: A top-level container object that aggregates a list of heterogeneous Feature objects under a single payload.
- Deep Answer: A FeatureCollection has a "type": "FeatureCollection" property and a "features" array. Each element in the array is an independent Feature containing its own geometry (Point, LineString, Polygon), an optional identifier, and a properties dictionary storing arbitrary metadata. This structure allows diverse spatial entities to be serialized and transmitted together.
- Planwise Example: Packaging the site boundary, non-developable areas, and tree markers into a single serialized collection for auto-save.
- Common Wrong Answer: Treating a FeatureCollection as a single geometry rather than iterating over its features array.

55. What is geometric self-intersection and why is it problematic?

- Short Answer: When a polygon's boundary edges cross over each other, creating an invalid "figure-eight" shape.
- Deep Answer: In planar geometry, a simple polygon must not intersect its own edges. When edges cross, the polygon's interior and exterior become mathematically ambiguous, creating "bowtie" topologies. Spatial databases (such as PostGIS) reject self-intersecting shapes as invalid (`ST_IsValid() == false`), causing area calculations and spatial clipping operations to throw errors.
- Planwise Example: Validating user-drawn parcel boundaries on the client to ensure edges do not cross before saving.
- Common Wrong Answer: Allowing users to save self-intersecting boundary polygons, which crashes downstream generative layout algorithms.

56. What is the winding order rule in RFC 7946?

- Short Answer: Exterior rings must follow a counter-clockwise direction; interior rings must follow a clockwise direction.
- Deep Answer: The right-hand rule in RFC 7946 specifies that when traversing a ring's vertices in order, the polygon's interior must always lie to the left. For exterior boundaries on a map, this corresponds to a counter-clockwise winding order. For interior rings (cutout holes), this corresponds to a clockwise winding order. While some display libraries tolerate incorrect winding, strict spatial analysis engines enforce it.
- Planwise Example: Normalizing coordinate winding orders before saving GeoJSON to prevent errors in downstream FSI calculations.
- Common Wrong Answer: Generating interior cutouts with counter-clockwise winding, which some spatial engines interpret as a second overlapping exterior boundary.

57. How do you determine whether a Point resides inside a Polygon?

- Short Answer: Using the Ray-Casting algorithm to count boundary crossings from the point to infinity.
- Deep Answer: The Ray-Casting algorithm projects an imaginary ray from the target point in any fixed direction (typically along the positive X-axis) out to infinity. It calculates how many times this ray intersects the polygon's boundary edges. If the number of intersections is odd, the point is inside the polygon; if even, the point is outside.
- Planwise Example: Verifying that tree marker points clicked on the canvas fall within the active parcel boundary.
- Common Wrong Answer: Assuming points located exactly on a polygon's boundary edge are cleanly inside; edge cases require explicit boundary tolerance handling.

58. What is the Haversine formula?

- Short Answer: An equation that calculates the great-circle distance between two points on a sphere given their longitudes and latitudes.
- Deep Answer: Given two coordinates $(\text{lat}_1, \text{lon}_1)$ and $(\text{lat}_2, \text{lon}_2)$ and the Earth's radius R : $\Delta \text{lat} = \text{lat}_2 - \text{lat}_1$, $\Delta \text{lon} = \text{lon}_2 - \text{lon}_1$ $a = \sin^2\left(\frac{\Delta \text{lat}}{2}\right) + \cos(\text{lat}_1)\cos(\text{lat}_2)\sin^2\left(\frac{\Delta \text{lon}}{2}\right)$ $c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$, $d = R \cdot c$ This accounts for the Earth's spherical curvature, providing accurate surface distances between coordinates.
- Planwise Example: Calculating the length of user-drawn measurement lines across parcels.
- Common Wrong Answer: Using basic Euclidean distance $(\sqrt{\Delta x^2 + \Delta y^2})$ on degree coordinates, which produces distorted distances that worsen at higher latitudes.

Level 8: High-Frequency Events, Debouncing, and Auto-Save (Questions 59–66)

59. What is debouncing?

- Short Answer: A programming pattern that delays executing a function until a specified

quiet period has elapsed since its last invocation.

- Deep Answer: When an event fires rapidly and repeatedly (such as dragging a polygon vertex across the canvas), debouncing ensures the target function runs only once after the activity pauses. Each new event resets a timer. The function executes only when the timer reaches its duration without any new events firing, preventing redundant operations.
- Planwise Example: Batching vertex adjustments during boundary drawing into a single auto-save API request sent 1000ms after editing stops.
- Common Wrong Answer: Confusing debouncing with throttling; debouncing delays execution until activity pauses, while throttling enforces a steady execution rate during activity.

60. What is throttling and when is it preferred over debouncing?

- Short Answer: Throttling limits function execution to a maximum of once per specified time interval during continuous activity.
- Deep Answer: While debouncing delays execution until events stop, throttling guarantees regular execution at fixed intervals while events are ongoing. Throttling is ideal for smooth UI updates that should respond during user interactions (e.g., updating calculated area numbers in the HUD as a polygon vertex is being dragged) without running on every single mouse frame.
- Planwise Example: Throttling on-screen area calculation updates to 10Hz during vertex dragging, while debouncing the backend auto-save API call.
- Common Wrong Answer: Using debouncing for visual HUD readouts, which leaves the area display static until the user releases the mouse.

61. How does a debounce function handle timer cancellation?

- Short Answer: By calling `clearTimeout(timerId)` whenever a new event occurs before the scheduled execution time.
- Deep Answer: The debounce function stores the timer identifier returned by `setTimeout` in a closure or persistent reference (`useRef`). Each time the debounced function is called, it checks for an existing timer ID and calls `clearTimeout(timerId)` to cancel the pending execution, then initializes a new `setTimeout` with the full delay.
- Planwise Example: Cancelling pending auto-save timers on new vertex movements so only the final shape is persisted.
- Common Wrong Answer: Storing the timer ID in standard component state (`useState`), which triggers unnecessary re-renders on every mouse movement.

62. What is an asynchronous race condition in an auto-save system?

- Short Answer: When an earlier, slower network request completes after a later, faster request, overwriting newer data with stale values.
- Deep Answer: If a user edits a shape, pauses (triggering Request 1), and then immediately edits it again (triggering Request 2), both requests may be in flight simultaneously. If Request 1 experiences network latency and resolves after Request 2, it writes its older, obsolete coordinates over the newer coordinates in the database, resulting in data loss.
- Planwise Example: Overwriting updated tree positions with older positions due to

overlapping auto-save requests.

- Common Wrong Answer: Assuming network requests always arrive and complete on the server in the exact order they were sent.

63. How does AbortController eliminate auto-save race conditions?

- Short Answer: By aborting pending in-flight HTTP requests before sending a new update request.
- Deep Answer: When a new auto-save request triggers, the application calls `.abort()` on the previous request's `AbortController`. The browser terminates the earlier HTTP connection immediately, ensuring that earlier request never completes or writes to the database, eliminating the risk of out-of-order writes.
- Planwise Example: Aborting pending geometry saves whenever a newer save payload is dispatched.
- Common Wrong Answer: Failing to catch the resulting `AbortError` in client error handlers, which can display confusing error alerts for intentionally cancelled requests.

64. What is Optimistic UI updating?

- Short Answer: Updating the user interface immediately under the assumption that the background server operation will succeed.
- Deep Answer: Instead of showing loading spinners and waiting for server confirmations, optimistic UI updates the visual interface instantly when the user acts. The application saves a snapshot of the previous state; if the server request ultimately fails, the application rolls the UI back to the snapshot and displays an informative error notification.
- Planwise Example: Rendering newly placed tree markers on the map instantly, persisting them via debounced auto-save in the background.
- Common Wrong Answer: Failing to save a rollback snapshot, leaving the UI out of sync with the database if the network request fails.

65. What is eventual consistency in web client architectures?

- Short Answer: A consistency model where client and server states may differ temporarily but are guaranteed to converge over time.
- Deep Answer: In applications with rapid user interactions and debounced saves, the client state changes continuously in memory while the server state lags behind. As long as every update eventually persists to the backend without conflicts, both systems converge to the same consistent state once editing pauses.
- Planwise Example: The Mapbox canvas reflects coordinate updates instantly, while the backend database converges to that state 1000ms after drawing stops.
- Common Wrong Answer: Assuming eventual consistency means you can ignore network failures; failed saves must notify the user and offer retry mechanisms.

66. How should offline state or network disconnects be handled during auto-save?

- Short Answer: By queuing failed mutations in local storage and re-synchronizing them when connectivity is restored.

- Deep Answer: The browser provides online and offline events. When offline, failed auto-save payloads can be placed into a persistent queue (in localStorage or IndexedDB) with a visual indicator. When connectivity returns, the queue is replayed sequentially to persist pending changes to the server.
- Planwise Example: Preventing data loss if a developer loses connection while digitizing parcel boundaries on a mobile hotspot.
- Common Wrong Answer: Silently dropping failed network updates, causing users to lose unsaved boundary edits on browser refresh.

Level 9: State Machines and UI Control Logic (Questions 67–74)

67. What is a Finite State Machine (FSM)?

- Short Answer: A computational model with a fixed set of states, transitioning from one state to another in response to specific events.
- Deep Answer: An FSM consists of a finite set of mutually exclusive states, a defined set of triggering events, transitions between states driven by events, and guard conditions that must be satisfied for transitions to occur. Modeling UI workflows with an FSM eliminates impossible states (e.g., trying to place trees before drawing a site boundary).
- Planwise Example: Governing toolbar behavior and disabling feature tools until a valid boundary polygon exists.
- Common Wrong Answer: Managing complex UI workflows with scattered boolean flags (isDrawing, hasBoundary, isTreeActive), leading to fragile and conflicting states.

68. What are guard conditions in state machine transitions?

- Short Answer: Boolean requirements that must evaluate to true before a state transition is allowed to execute.
- Deep Answer: Even if an event occurs, a guard condition can block the transition if prerequisite rules are not met. If the guard evaluates to false, the system remains in its current state, keeping the interface predictable.
- Planwise Example: Attempting to select the tree placement tool triggers a guard that checks `hasBoundary === true`; if false, the transition is blocked.
- Common Wrong Answer: Disabling buttons visually without also guarding the underlying event handlers, allowing keyboard shortcuts to trigger invalid transitions.

69. What is derived state in React and why is it preferred over duplicated state?

- Short Answer: Values calculated on the fly from existing state or props during rendering, rather than stored in separate state variables.
- Deep Answer: Storing calculated values in separate state variables requires keeping them in sync across renders, which easily causes desynchronization bugs. Derived state computes values dynamically during rendering (using `useMemo` for expensive calculations). This guarantees calculations always reflect the latest underlying state.
- Planwise Example: Calculating `toolActivationRules` directly from `hasBoundary` during render rather than maintaining parallel state variables.
- Common Wrong Answer: Using a `useEffect` to copy a prop into a local state variable, creating duplicate state that can become stale.

70. How does a centralized toolActivationRules object simplify toolbar maintenance?

- Short Answer: It consolidates all tool enable/disable rules in a single place instead of scattering conditions across individual button components.
- Deep Answer: Centralizing rules into an object creates a single source of truth for tool availability. Individual buttons simply read their corresponding flag from this object. Updating workflow rules requires editing one centralized object rather than hunting down conditions across multiple files.
- Planwise Example: The Toolbar component reads toolActivationRules to apply disabled classes and control tool accessibility consistently.
- Common Wrong Answer: Hardcoding conditional expressions directly inside individual JSX button elements throughout the component tree.

71. How do you implement an accessible disabled state on a custom SVG toolbar button?

- Short Answer: Add the HTML disabled attribute, set aria-disabled="true", and provide an explanatory title or tooltip.
- Deep Answer: Custom UI buttons require proper accessibility markup. Simply styling a button with lower opacity does not convey its disabled status to assistive technologies. An accessible disabled button uses the native disabled attribute (for <button> elements), sets aria-disabled="true", disables pointer events via CSS (pointer-events: none), and provides a descriptive title or aria-label explaining why the tool is unavailable.
- Planwise Example: Disabled tree buttons display the tooltip: "Draw site boundary first to enable tree placement".
- Common Wrong Answer: Using CSS pointer-events: none alone, which removes mouse interactions but leaves the button focusable and clickable via keyboard navigation.

72. What is the difference between a modal dialog and an overlay?

- Short Answer: A modal interrupts workflow and requires interaction before returning to the page; an overlay displays supplementary content over the existing view.
- Deep Answer: A modal dialog creates a focused interaction mode: it renders over the main view with a dark backdrop, traps keyboard focus inside itself, disables background page scrolling, and requires explicit dismissal. An overlay presents information alongside the primary view without interrupting interaction with the underlying canvas.
- Planwise Example: The GetRegulatoryForm displays as a focused modal dialog; the area metrics HUD floats as a non-blocking overlay.
- Common Wrong Answer: Failing to trap keyboard focus inside a modal dialog, allowing keyboard users to tab to invisible background elements.

73. How do you prevent layout shifts when dynamic error messages appear in forms?

- Short Answer: Reserve vertical space for error messages in the CSS layout or use absolute positioning within a padded container.

- Deep Answer: When validation error text appears dynamically below an input field, it pushes down all content below it, causing cumulative layout shift (CLS). This can be prevented by reserving space (e.g., setting a min-height on error containers) or positioning error text absolutely within a container with fixed bottom padding, keeping the form layout stable.
- Planwise Example: Reserving space below form fields in NewProject so inline error messages do not jump the submit button.
- Common Wrong Answer: Letting dynamic error text push the primary submit button off the bottom of the screen on smaller displays.

74. What is a CSS backdrop filter and how does it improve map overlay readability?

- Short Answer: A CSS property that applies graphical effects (like blurring) to the area behind an element.
- Deep Answer: Floating UI panels on top of satellite map imagery can be difficult to read due to high-contrast terrain details behind them. Applying backdrop-filter: blur(8px) with a semi-transparent background color diffuses the underlying map imagery, keeping the text legible while preserving the visual context of the map underneath.
- Planwise Example: Styling the floating Toolbar and HUD metrics panels on MapPage with backdrop blurring for legibility over satellite tiles.
- Common Wrong Answer: Using backdrop filters without testing performance on lower-end devices, where heavy CSS blurring can cause frame rate drops.

Level 10: Performance Optimization and System Scalability (Questions 75–82)

75. How does React.memo prevent unnecessary component re-renders?

- Short Answer: It wraps a component to skip re-rendering if its incoming props have not changed under shallow comparison.
- Deep Answer: By default, when a parent component re-renders, all of its children re-render automatically. Wrapping a child component in React.memo instructs React to compare its new props against its previous props using shallow equality. If all props are identical, React skips rendering the child, reusing its existing virtual DOM tree.
- Planwise Example: Memoizing the Toolbar component so it does not re-render when the HUD metrics panel updates on MapPage.
- Common Wrong Answer: Passing un-memoized object literals or inline arrow functions as props to a memoized component, which breaks memoization because new references are created on every render.

76. What is the difference between useMemo and useCallback?

- Short Answer: useMemo caches the result of a calculation; useCallback caches a function definition between renders.
- Deep Answer: Both hooks preserve references across renders based on a dependency array:

- `const result = useMemo(() => computeExpensiveValue(a, b), [a, b]);`
- `const fn = useCallback(() => { doSomething(a); }, [a]);` Under the hood, `useCallback(fn, deps)` is equivalent to `useMemo(() => fn, deps)`. They prevent expensive recalculations and provide stable callback references to memoized children.
- Planwise Example: Using `useCallback` on `handleDrawChange` to keep its function reference stable for Mapbox event listeners.
- Common Wrong Answer: Wrapping inexpensive operations in `useMemo`, which adds hook overhead without providing any measurable performance benefit.

77. How does client-side list pagination improve rendering performance?

- Short Answer: It limits how many DOM elements are rendered simultaneously, reducing browser memory usage and layout reflow times.
- Deep Answer: Rendering hundreds of complex cards with images and action buttons creates thousands of DOM nodes. This bloats browser memory and slows down layout recalculations. Slicing the data array (e.g., rendering an initial 7 cards and appending 4 more on "Load More") keeps the active DOM tree small, maintaining fast rendering and smooth scrolling.
- Planwise Example: The Projects dashboard renders an initial 7 projects and appends 4 on each "Load More" click.
- Common Wrong Answer: Slicing arrays only on the client for huge datasets (thousands of records), which still requires downloading the full dataset over the network; large datasets need server-side pagination.

78. What is code-splitting and dynamic importing with `React.lazy()`?

- Short Answer: Splitting a JavaScript bundle into smaller chunks that load on demand when specific routes or components are visited.
- Deep Answer: By default, bundlers pack all application code into a single large JavaScript file. Heavy dependencies (such as `mapbox-gl`, which can exceed 500KB) slow down initial page loads. `React.lazy(() => import('./MapPage'))` splits that component into a separate bundle chunk that downloads only when the user navigates to `/map`, accelerating initial dashboard loading.
- Planwise Example: Lazy-loading the heavy `MapPage` route while keeping the initial `/login` bundle lightweight.
- Common Wrong Answer: Using `React.lazy` without wrapping the component in a `<Suspense>` boundary with a fallback loader, which causes React to throw an error during loading.

79. What causes memory leaks in Single Page Applications and how are they prevented?

- Short Answer: Dangling event listeners, uncleared timers, and retained object references that prevent the garbage collector from freeing memory.
- Deep Answer: Because an SPA never reloads the HTML page, objects retained in memory accumulate over time. Common sources include:
 1. `setInterval` or `setTimeout` handles that are not cleared on unmount.

- 2. Global event listeners (`window.addEventListener`) that are not removed.
- 3. External library instances (like Mapbox) whose `.remove()` or `.destroy()` methods are never called. Cleaning up these resources inside `useEffect` return functions prevents memory accumulation.
- Planwise Example: Calling `map.remove()` and clearing debounce timers when unmounting `MapPage`.
- Common Wrong Answer: Adding an event listener inside a component without removing it in the cleanup return function, causing duplicate listeners to accumulate on every render.

80. How do you profile React component rendering performance?

- Short Answer: Using the React DevTools Profiler to record component render durations, frequencies, and what triggered each render.
- Deep Answer: The React Profiler records component rendering activity during interactions. It displays flame charts and ranked charts showing how long each component took to render and provides details on what caused each render (e.g., specific prop changes, hook updates, or parent re-renders). This allows developers to pinpoint slow components and eliminate wasteful re-renders.
- Planwise Example: Profiling `MapPage` during vertex dragging to verify that canvas operations are not triggering re-renders in surrounding form components.
- Common Wrong Answer: Profiling applications in development mode rather than production mode; development builds include debug overhead that skews timing data.

81. What is tree-shaking and what enables it in modern build systems?

- Short Answer: An optimization that removes unused JavaScript code from the final bundle; enabled by static ES module syntax.
- Deep Answer: Bundlers (Webpack, Vite, Rollup) analyze ES module import and export statements statically at build time. If an imported library exports 50 utility functions but the application references only two, the bundler excludes the 48 unreferenced functions from the compiled production bundle, significantly reducing download sizes.
- Planwise Example: Importing only specific geometric utilities from `Turf.js` rather than the entire library bundle.
- Common Wrong Answer: Using wildcard imports (`import * as Turf from '@turf/turf'`), which can prevent the bundler from eliminating unused functions.

82. What are Web Workers and how could they optimize heavy spatial calculations?

- Short Answer: Background JavaScript threads that run heavy computational tasks without freezing the browser's main UI thread.
- Deep Answer: JavaScript on the main thread shares execution time with UI layout, user inputs, and rendering animations. Heavy computational tasks (such as running spatial clipping algorithms on thousands of boundary points) freeze the interface. Offloading these calculations to a Web Worker executes the math on a separate CPU thread, communicating results back via messages and keeping the UI smooth at 60 frames per second.
- Planwise Example: Running intensive 3D massing and generative layout calculations

inside a Web Worker while the user interacts with the map.

- Common Wrong Answer: Passing DOM elements or WebGL contexts directly to a Web Worker; Workers do not have access to the DOM or main window context.

Level 11: Domain-Specific Real Estate Feasibility Questions (Questions 83–92)

83. What is Floor Space Index (FSI) / Floor Area Ratio (FAR)?

- Short Answer: The ratio of the total permissible built-up floor area of a building to the gross area of the land parcel.
- Deep Answer: If a land parcel measures 1,000 square meters and the municipal development code grants an FSI of 3.0, the developer can construct a maximum of 3,000 square meters of gross built-up floor area across all floors: $\text{FSI} = \frac{\text{Total Built-Up Area}}{\text{Plot Area}}$ Permissible FSI varies based on municipal zoning, adjacent road widths, and applicable development schemes (e.g., DCPR 2034 in Mumbai).
- Planwise Example: Captured via regulatory forms to calculate maximum development envelopes for digitized parcels.
- Common Wrong Answer: Confusing gross permissible FSI with net carpet saleable area; efficiency losses reduce the actual saleable area.

84. Why does plot shape significantly impact land efficiency?

- Short Answer: Irregular plots require larger perimeter setbacks, reducing the buildable footprint and worsening core-to-floorplate ratios.
- Deep Answer: Rectangular parcels allow efficient, regular building footprints with compact structural grids and centralized circulation cores, often achieving 90–92% spatial efficiency. Irregular, triangular, or sharply tapered plots force elongated or fragmented footprints. These irregular footprints require longer corridors, more structural shear walls, and larger circulation areas, reducing saleable efficiency down to 80–85%. This efficiency drop directly reduces net revenue and project profitability.
- Planwise Example: Visualizing irregular parcel geometries on the map to evaluate whether a plot can support an efficient building footprint.
- Common Wrong Answer: Assuming two plots with identical square footage have the same development value without evaluating their geometry and setbacks.

85. How does adjacent road width dictate permissible building height in Mumbai regulations?

- Short Answer: Narrower roads cannot support high-capacity fire engines, so regulations cap building heights and limit FSI loading on narrow roads.
- Deep Answer: Under municipal codes (like DCPR 2034 and UDCPR), permissible building height is directly linked to the width of the fronting access road. A 9-meter road imposes strict height limits and restricts premium FSI or Transferable Development Rights (TDR) loading. An 18-meter or 24-meter road unlocks high-rise classifications, allowing taller towers with slimmer, more efficient vertical cores and higher FSI consumption.

- Planwise Example: The GetRegulatoryForm captures road width as a core input to determine permissible heights and development schemes.
- Common Wrong Answer: Evaluating a parcel's FSI potential without verifying that the adjacent road width can legally support that density.

86. What are mandatory municipal setbacks and how do they eat into buildable footprints?

- Short Answer: Statutory open spaces required around the perimeter of a building where no permanent construction is permitted.
- Deep Answer: Building codes require open spaces along the front, rear, and sides of a plot for fire tender access, ventilation, daylighting, and emergency egress. These setbacks are calculated based on plot size, road width, and proposed building height. On a narrow 25-meter-wide plot, mandatory 6-meter side setbacks consume 12 meters of total width, leaving an unviable 13-meter-wide buildable strip.
- Planwise Example: Subtracting setback margins from the digitized boundary to calculate the net developable envelope.
- Common Wrong Answer: Treating the gross digitized parcel boundary as buildable land without offsetting for mandatory setbacks.

87. What is the difference between Gross Plot Area and Net Developable Area?

- Short Answer: Gross Plot Area is the total land within the boundary; Net Developable Area subtracts setbacks, deductions, and exclusion zones.
- Deep Answer: Gross Plot Area is the entire surveyed property perimeter. Net Developable Area subtracts all non-buildable portions: mandatory road-widening reservations, utility easements, steep slopes, preserved environmental zones, and mandatory perimeter setbacks. Financial returns and actual building footprints are governed entirely by the Net Developable Area.
- Planwise Example: The HUD metrics panel dynamically displays both numbers as non-developable areas are drawn within the boundary.
- Common Wrong Answer: Calculating project revenues using the Gross Plot Area rather than the Net Developable Area.

88. What are Transferable Development Rights (TDR)?

- Short Answer: Development rights granted as certificates to landowners whose land is acquired for public infrastructure, which can be sold or used elsewhere.
- Deep Answer: When a municipality acquires private land for public purposes (road widening, parks, transit corridors) without cash compensation, it awards the owner TDR certificates. These certificates allow the owner to build additional floor area beyond the base FSI on an eligible receiving plot, or sell those development rights to other developers in designated zones.
- Planwise Example: Capturing road-widening deductions that may qualify a parcel for compensatory TDR generation.
- Common Wrong Answer: Assuming TDR can be loaded onto any plot regardless of location; loading potential is restricted by receiving zone rules and access road widths.

89. Why are mature trees modeled as point constraints with radial buffers?

- Short Answer: Environmental laws protect mature trees from felling, requiring construction to stay clear of their root systems and canopies.
- Deep Answer: Municipal tree protection authorities prohibit the arbitrary felling of mature trees. Preserving a tree requires leaving a buffer zone clear of excavation to protect its root system and canopy. In Planwise, trees are mapped as point features that establish circular exclusion buffers, preventing building foundations from encroaching into the protected root zone.
- Planwise Example: The Tree tool places point markers inside the parcel to factor environmental preservation into the buildable footprint.
- Common Wrong Answer: Treating tree preservation as a simple point coordinate without accounting for the required radial root protection buffer.

90. What is the difference between Private Freehold Land and Collector Land?

- Short Answer: Freehold land grants absolute ownership; Collector Land is government land leased with transfer restrictions and revenue-sharing rules.
- Deep Answer: Freehold land grants full ownership with no ongoing lease payments to the revenue department. Collector Land (Class II or Leasehold) is owned by the state government and leased to occupants. Developing or transferring Collector Land requires prior sanction from the District Collector, payment of substantial unearned income shares or transfer premiums, and adherence to specific redevelopment terms.
- Planwise Example: The NewProject dropdown captures land ownership type because it directly dictates statutory approval costs and timelines.
- Common Wrong Answer: Assuming private freehold land and collector land follow the same approval workflows and development timelines.

91. What is the role of MHADA Land in urban redevelopment?

- Short Answer: Land owned by the Maharashtra Housing and Area Development Authority, governed by dedicated redevelopment regulations (e.g., Regulation 33(5)).
- Deep Answer: MHADA developments consist of public housing colonies constructed across previous decades. Redevelopment is governed by dedicated regulatory schemes (such as Regulation 33(5) under DCPR 2034) that grant incentive FSI to developers in exchange for constructing rehabilitation housing for existing occupants and handing over surplus housing stock or premium shares to MHADA.
- Planwise Example: Selecting "MHADA Land" in the project creation form applies specific redevelopment FSI rules to the feasibility analysis.
- Common Wrong Answer: Modeling MHADA land redevelopment with the same regulatory assumptions as private vacant greenfield parcels.

92. Why is parking configuration (Basement vs. Podium) a critical business constraint?

- Short Answer: Basements require expensive deep excavation and waterproofing, while above-ground podiums consume vertical height and alter tower massing.

- Deep Answer: Parking requirements are dictated by municipal codes based on unit counts and commercial floor area. Providing this parking via subterranean basements requires deep excavation, shoring, and waterproofing, which dramatically increases structural costs and lengthens construction schedules. Above-ground parking podiums are faster and cheaper to construct, but they consume vertical height allowances and can compromise street-level retail design.
- Planwise Example: The BusinessConstraints form captures parking typology to evaluate structural cost and massing tradeoffs early in feasibility.
- Common Wrong Answer: Treating parking design as an afterthought; in high-density urban developments, parking layouts can make or break site feasibility.

Level 12: Production System Architecture and System Design (Questions 93–100)

93. How would you design a spatial caching layer for parcel boundaries?

- Short Answer: Use a CDN for static map assets, an in-memory Redis cache for GeoJSON features, and spatial bounding-box queries in the database.
- Deep Answer: A multi-tier caching architecture optimizes spatial data retrieval:
 1. Edge Layer: Static vector tiles are cached globally via a Content Delivery Network (Cloudflare/CloudFront).
 2. Application Cache: Frequently accessed project GeoJSON features are stored in an in-memory Redis cluster keyed by project ID.
 3. Client Layer: React Query caches GeoJSON responses in browser memory, eliminating redundant network calls when switching tabs.
 4. Spatial Index: The database maintains PostGIS R-Tree spatial indices (GIST) to execute fast bounding-box queries (ST_Intersects).
- Planwise Example: Caching project boundary geometries so returning to MapPage from the dashboard displays the parcel instantly.
- Common Wrong Answer: Caching spatial data without invalidating the cache when users edit geometries, leading to stale map views.

94. How do you prevent overlapping polygon drawing when digitizing parcels?

- Short Answer: Run client-side boolean geometry checks using spatial algorithms to detect and prevent boundary overlaps before saving.
- Deep Answer: When multiple parcels are digitized adjacent to each other, users can accidentally overlap boundaries. To prevent this, the client executes boolean intersection checks (via Turf.js `turf.intersect` or Martinez-Rueda clipping algorithms) against existing adjacent parcel layers. If the intersection polygon area exceeds zero, the editor rejects the newly placed vertex and highlights the overlapping boundary in red.
- Planwise Example: Preventing drawn non-developable area cutouts from extending outside the primary parcel boundary.
- Common Wrong Answer: Relying solely on visual inspection on the map canvas; slight overlaps that are invisible at high zoom levels will still break spatial database queries.

95. What is the 12-Factor App methodology and how does it apply to Planwise?

- Short Answer: A set of twelve best practices for building scalable, maintainable, cloud-native software applications.
- Deep Answer: Key factors applied to this architecture include:
 - Config (Factor 3): Store configuration in the environment (REACT_APP_MAPBOX_TOKEN, API URLs), never in source code.
 - Backing Services (Factor 4): Treat the API backend and databases as attached network resources.
 - Stateless Processes (Factor 6): Keep API services stateless, relying on JWTs and databases for state.
 - Port Binding (Factor 7): The backend exports services by binding directly to port 5001.
 - Disposability (Factor 9): Fast startup and graceful shutdown for frontend build containers and backend microservices.
- Planwise Example: Externalizing API endpoints and Mapbox tokens into environment configuration files.
- Common Wrong Answer: Hardcoding API URLs like `http://localhost:5001` directly in component source files rather than reading them from environment variables.

96. How would you design a collaborative real-time editing architecture for Planwise?

- Short Answer: Use WebSockets combined with Conflict-free Replicated Data Types (CRDTs) to synchronize concurrent map edits in real time.
- Deep Answer: A collaborative architecture replaces HTTP polling with WebSockets:
 1. Transport: A persistent bidirectional WebSocket connection connects clients to an operational server.
 2. Conflict Resolution: Instead of simple last-write-wins (which overwrites concurrent edits), the system uses CRDTs (such as Yjs or Automerge) to model spatial coordinates. When two planners adjust different vertices of a boundary simultaneously, the CRDT merges both edits deterministically without conflicts.
 3. Presence: User cursor positions and active tool selections are broadcast as lightweight messages, rendering live user avatars on the map.
- Planwise Example: Allowing an acquisition manager and an architect to review and edit the same parcel boundary simultaneously from different locations.
- Common Wrong Answer: Attempting to implement real-time collaborative editing using raw REST polling, which produces high latency and frequent data overwrites.

97. What is Continuous Integration and Continuous Deployment (CI/CD)?

- Short Answer: An automated pipeline that builds, tests, and deploys code changes to production environments on every repository commit.
- Deep Answer: CI/CD automates the release process:
 1. Continuous Integration: On every pull request, automated runners execute linting (ESLint), code formatting checks (Prettier), and automated test suites (unit tests via Vitest/Jest, end-to-end tests via Playwright).

2. Continuous Deployment: Once tests pass and the PR merges to the main branch, the pipeline automatically compiles production bundles, runs tree-shaking, and deploys static assets to edge CDN storage while updating backend container images.
- Planwise Example: Automated pipelines validating that React components compile cleanly and meet ESLint standards before merging feature branches.
 - Common Wrong Answer: Skipping automated testing in the CI pipeline, allowing broken builds to deploy directly to staging or production.

98. How do you implement end-to-end (E2E) testing for a WebGL map canvas?

- Short Answer: Use tools like Playwright to simulate user interactions, verify network API payloads, and compare canvas screenshot snapshots.
- Deep Answer: Testing WebGL canvases is challenging because shapes drawn inside the canvas do not exist as distinct HTML DOM elements. A complete E2E testing strategy uses three complementary techniques:
 1. Interaction Simulation: Playwright dispatches real mouse clicks at specific canvas pixel offsets to draw shapes.
 2. Network Inspection: Intercept and assert against the resulting GeoJSON payloads sent in auto-save API calls.
 3. Visual Regression: Capture canvas screenshots and run pixel-diff comparisons against reference baseline images to catch rendering anomalies.
- Planwise Example: An automated test that logs in, creates a test project, clicks four canvas points to draw a parcel, and asserts that the auto-save PUT request was sent with valid GeoJSON.
- Common Wrong Answer: Trying to query canvas shapes using standard DOM selectors like `document.querySelector('.polygon')`; canvas contents are drawn pixels, not DOM nodes.

99. What are Content Security Policies (CSP) and what directives does Mapbox require?

- Short Answer: An HTTP header that restricts which external resources (scripts, styles, connections) the browser is allowed to load.
- Deep Answer: A Content Security Policy (CSP) protects against XSS attacks by restricting where scripts, styles, images, and network connections can originate. Mapbox GL JS relies on Web Workers, WebGL blobs, and external vector tile servers, requiring specific CSP directives:
 - `connect-src 'self' https://api.mapbox.com https://*.tiles.mapbox.com;`
 - `worker-src 'self' blob;;`
 - `child-src blob;;`
 - `img-src 'self' data: blob: https://api.mapbox.com;` Omitting these directives causes the browser to block map tiles and Web Worker initialization.
- Planwise Example: Configuring the web server's CSP headers so Mapbox can load satellite tiles and run its rendering worker threads securely.
- Common Wrong Answer: Setting a strict default-src 'self' policy without adding Mapbox exception domains, which silently prevents maps from loading.

100. If you were rebuilding Planwise from scratch today, what would you change and why?

- Short Answer: Migrate to TypeScript, adopt React Query for state management, use an isolated spatial drawing engine, and implement secure HttpOnly cookie authentication.
- Deep Answer: Four key architectural upgrades would elevate the platform from an early-stage prototype to an enterprise-grade production system:
 1. Type Safety: Adopt TypeScript across the entire codebase. Defining strict interfaces for GeoJSON features, regulatory parameters, and API responses catches coordinate inversions and schema mismatches at compile time.
 2. State Architecture: Replace scattered useState hooks with TanStack Query (React Query) for server state and caching, paired with a lightweight Zustand store for client-only UI states (like active drawing tools).
 3. Isolated Map Architecture: Decouple Mapbox GL JS from React rendering entirely using a dedicated canvas controller class. This prevents map re-renders, simplifies lifecycle management, and isolates canvas operations from React component lifecycles.
 4. Enterprise Security: Replace localStorage token storage with secure, HttpOnly, SameSite=Strict cookies paired with automated refresh token rotation to neutralize XSS vulnerabilities.
- Planwise Example: Demonstrating technical maturity in an interview by recognizing early implementation constraints and presenting clear architectural upgrades.
- Common Wrong Answer: Claiming an early-stage project is already perfect; seasoned engineers demonstrate their expertise by identifying limitations and explaining how they would improve the architecture.

Part 20: Step-by-Step Rebuild Roadmap

This structured 16-stage engineering roadmap outlines the technical milestones required to construct the platform from an empty repository up to an enterprise-grade production deployment.

1. Stage 0: Tooling & Environment Setup: Establish repository structure, Node.js LTS runtime, .gitignore, ESLint, Prettier, and environment variable handlers.
2. Stage 1: React 18 Application Scaffolding: Configure build tooling, modular SCSS architecture, and core application shell.
3. Stage 2: Authentication Core & JWT Session Management: Build Login component, credential validation, visibility toggles, and token storage.
4. Stage 3: Routing Architecture & Protected Route Wrappers: Configure React Router, client route guards, and navigation state preservation.
5. Stage 4: Project CRUD Dashboard & Progressive Loading: Build Projects view, progressive 7+4 card pagination, and deletion handlers.
6. Stage 5: Mapbox GL JS Integration & WebGL Canvas Lifecycle: Initialize canvas inside useRef, configure controls, and integrate India-restricted geocoder.
7. Stage 6: GeoJSON Data Layer & Geometry Validation: Implement pure spatial validation functions for ring closure, self-intersections, and winding order.
8. Stage 7: Mapbox Draw Integration & Custom Controls: Bind draw event listeners,

- synchronize vector geometry with React, and build HUD metrics panel.
9. Stage 8: Environmental Tools & Tree Marker Infrastructure: Add point marker modes for trees and dedicated polygon tools for non-developable cutouts.
 10. Stage 9: Regulatory & Business Constraint Form Overlays: Implement modal forms (GetRegulatoryForm, BusinessConstraints) with client-side validation.
 11. Stage 10: Dynamic Toolbar & State Machine Rules: Build floating Toolbar driven by centralized toolActivationRules.
 12. Stage 11: Debounced Auto-Save & Mutation Queue: Implement debounced auto-save hook with useRef timer handling and AbortController cancellation.
 13. Stage 12: Backend REST Integration & Payload Alignment: Connect Axios client to port 5001, formatting payloads to match documented schema.
 14. Stage 13: Layered Error Handling & User Feedback: Integrate React Toastify, inline form error highlights, and WebGL error boundaries.
 15. Stage 14: Comprehensive Testing Suite: Author unit tests for geometry validation, component tests for forms, and Playwright E2E workflows.
 16. Stage 15: Production Hardening & Deployment: Configure production bundling, lazy loading, CSP security headers, and Docker containerization.

Final Synthesis Deliverables

Deliverable 1: One-Page Project Cheat Sheet

- Project Name: Archonet Planwise Site Development System.
- Organization: Archonet (Bengaluru, India).
- Primary Tech Stack: React.js, React Router, Mapbox GL JS, Mapbox Draw, Mapbox Geocoder, Axios, React Toastify, CSS Modules/SCSS.
- Backend Interface: RESTful API listening on <http://localhost:5001/api/projects> using Bearer JWT authentication.
- Core Purpose: An interactive spatial web platform that digitizes land parcels, enforces municipal and business constraints, and evaluates site feasibility in minutes.
- Key Frontend Components:
 - Login: Authenticates users and stores JWT in localStorage.
 - Projects: Paginated, searchable dashboard with progressive "Load More" loading (initial 7, incrementing by 4).
 - NewProject: Controlled modal capturing project title, description, and land ownership.
 - ProjectSelection: Intermediate workflow card picker (Site Feasibility, Residential Unit Layout, Building Massing).
 - MapPage: Interactive WebGL spatial workspace with geocoding, polygon drawing, and real-time HUD metrics.
 - Toolbar: Dual-sided floating tool palette driven by toolActivationRules (enforcing boundary creation before tree/deduction tools activate).
 - GetRegulatoryForm: Captures municipal parameters (road width, setbacks, height limits).
 - BusinessConstraints: Captures commercial targets (product mix, parking preferences).
 - GetBusinessConstraints: Retrieves and displays previously saved commercial

- constraints.
 - InputSummary: Consolidates site geometries and constraints into a pre-flight review before continuing to /site-constraints.
- Project Creation Payload Contract: { project_title, about, land_ownership, geojson: "{}", nondevelopableareas: "[]", trees: "[]" }.
- Key Engineering Highlights:
 - Managing imperative Mapbox GL JS WebGL instances inside React useRef to prevent unnecessary re-renders.
 - 1000ms debounced auto-save with request cancellation to prevent backend flooding and race conditions.
 - Finite State Machine logic governing toolbar tool activation based on parcel boundary presence.

Deliverable 2: Complete System Architecture Matrix

Architectural Layer	Core Technology	Primary Responsibilities	Data Inputs	Data Outputs	Documented vs Inferred Status
Presentation	Browser (Blink/V8)	Renders HTML5 DOM, manages WebGL canvas context, captures pointer and keyboard interactions.	User clicks, coordinate inputs, form typing.	Native DOM events (click, change, pointerdown).	Explicitly Documented
Component Shell	React 18 SPA	Declarative component tree: forms, dashboards, dynamic toolbars, review overlays.	DOM events, component props, React Context.	Virtual DOM updates, hook state changes, HTTP triggers.	Explicitly Documented
State & Ref Layer	React Hooks	Manages form state, modal visibility flags, HUD metrics, and preserves mutable Mapbox instances.	Input values, API responses, Mapbox Draw events.	Cleaned state objects, debounced update signals.	Explicitly Documented
Geospatial Engine	Mapbox GL JS & Draw	WebGL vector map rendering, polygon drawing, tree point markers,	Pointer click coordinates, vector map tiles, GeoJSON.	WGS84 GeoJSON features, bounding boxes, camera	Explicitly Documented

Architectural Layer	Core Technology	Primary Responsibilities	Data Inputs	Data Outputs	Documented vs Inferred Status
		geodesic calculations.		angles.	
Transport Layer	Axios HTTP Client	Asynchronous Promise-based networking, request serialization, debounced auto-save queuing.	JavaScript objects, GeoJSON text, target URLs.	Serialized HTTP requests (application/json).	Explicitly Documented
Security Gateway	JWT in localStorage	Injects Authorization: Bearer <token> into API headers; guards protected routes.	User credentials, stored JWT token strings.	Authorized HTTP request headers, route permissions.	Explicitly Documented
Backend REST API	REST on Port 5001	Validates incoming payloads, parses stringified GeoJSON, enforces business logic, manages controllers.	HTTP requests with project parameters and GeoJSON.	JSON response objects (200 OK, 201 Created, etc.).	Documented Endpoints / Inferred Node/Express
Persistence Store	Relational / Document	Stores user credentials, project metadata, constraints, and spatial geometries.	SQL insert statements or BSON document records.	Persisted entity records, spatial query results.	Unknown Implementation

Deliverable 3: Complete User Workflow

1. Access Gateway: User navigates to /login, enters email and password, and submits credentials.
2. Session Establishment: Backend returns signed JWT; client persists token in localStorage and redirects to /home.
3. Dashboard Management: Projects dashboard loads; user browses paginated cards or clicks "Create New Project".
4. Project Initialization: NewProject modal opens; user inputs title, description, and selects land ownership category.

5. Backend Entity Creation: POST /api/projects returns 201 Created; project record is saved; user selects project tile.
6. Workflow Intent Selection: ProjectSelection view opens; user chooses "Site Feasibility" workflow card, routing to /map.
7. Geographic Location Search: MapPage initializes WebGL canvas; user searches site location via India-restricted geocoder.
8. Parcel Boundary Delineation: User activates boundary tool on Toolbar, draws polygon perimeter; HUD metrics update area.
9. Feature & Constraint Placement: Toolbar enables environmental tools; user marks preserved trees and outlines exclusion zones.
10. Debounced Background Save: Drawing pauses; 1000ms debounce timer completes; auto-save PUT persists geometries to backend.
11. Regulatory & Business Form Completion: User opens GetRegulatoryForm, enters road widths and setbacks, and completes BusinessConstraints form.
12. Pre-Flight Review & Progression: InputSummary modal displays consolidated review table; user confirms and navigates to /site-constraints.

Deliverable 4: Complete Data Flow

1. User Canvas Interaction: User clicks vertices to draw boundary on Mapbox canvas.
2. Mapbox Draw Module: Captures mouse clicks and normalizes them into WGS84 coordinate pairs.
3. Mapbox Event Bus: Dispatches draw.create or draw.update events containing raw GeoJSON features.
4. React MapPage Container: Extracts features, validates polygon closure, and updates HUD area metrics.
5. Debounce Controller: Resets 1000ms countdown timer; aborts pending in-flight HTTP requests via AbortController.
6. Data Serialization Pipeline: Serializes spatial data structures into JSON strings (JSON.stringify(draw.getAll())).
7. Axios HTTP Client: Retrieves JWT from localStorage and injects Authorization: Bearer <token> header.
8. Network Transport: Dispatches HTTP PUT request with geometry payload to http://localhost:5001/api/projects/:id.
9. Backend API Controller: Validates token signature, unpacks stringified GeoJSON, and executes schema checks.
10. Persistence Layer: Saves project geometries and constraints into database transaction.
11. HTTP Response Cycle: Returns HTTP 200 OK confirmation response to client.
12. UI Feedback: Displays toast notification confirming auto-save and clears saving indicator.

Deliverable 5: Component Responsibility Matrix

Component Name	Core Architectural Purpose	Documented UI Controls & Elements	Outgoing API Calls & Endpoints	Downstream Transitions
Login	Authenticates users and	Email and password inputs,	POST /api/auth/login	On success: writes token to

Component Name	Core Architectural Purpose	Documented UI Controls & Elements	Outgoing API Calls & Endpoints	Downstream Transitions
	establishes local session.	password visibility toggle, submit button.		localStorage, navigates to /home.
Projects	Paginated catalog for browsing, searching, and managing parcels.	Search filter, project cards, "Create New Project" tile, "Load More" button, delete buttons.	GET /api/projects DELETE /api/projects/:id	Tile click: opens ProjectSelection Create click: opens NewProject modal.
NewProject	Captures primary legal and administrative parameters for parcels.	Text inputs for title and description, land ownership dropdown with 5 options, submit button.	POST http://localhost:5001/api/projects	On 201 Created: closes modal, refreshes dashboard list, provides project context.
ProjectSelection	Intermediate routing card picker for planning workflows.	Three workflow cards: Site Feasibility, Residential Unit Layout, Building Massing.	None (routes based on navigation state).	Feasibility click: navigates to /map Other cards: navigate to downstream workspaces.
MapPage	Main spatial workspace coordinating canvas, drawing, and metrics.	WebGL canvas, HUD metrics display panel, geocoder search input, modal containers.	PUT /api/projects/:id (debounced auto-save)	Review confirmation: navigates to /site-constraints.
Toolbar	Floating palette managing drawing tools, perspectives, and modals.	Boundary, tree, deduction, measurement, grid, 3D view, camera reset, and modal buttons.	POST /api/projects/:id/upload-image (file upload)	Drives Mapbox Draw operational modes and toggles form modal overlays.
GetRegulatoryForm	Modal capturing municipal planning rules for the parcel.	Inputs for road width, front/rear/side setbacks, permissible height limit.	POST /api/projects/:id/regulatory	On save: updates parent project state, provides data for InputSummary.
BusinessConstraints	Modal capturing commercial objectives and product typologies.	Product mix checkboxes (Residential, Retail), parking radio controls,	POST /api/projects/:id/business	On save: updates parent project state, opens InputSummary review.

Component Name	Core Architectural Purpose	Documented UI Controls & Elements	Outgoing API Calls & Endpoints	Downstream Transitions
		carpet area inputs.		
GetBusinessConstraints	Fetch-and-display overlay for reviewing saved business parameters.	Read-only constraint cards, "Edit" button to toggle inputs, close button.	GET /api/projects/:id/business	Edit click: switches to editable form fields; close returns to map.
InputSummary	Pre-flight review consolidating geometries and constraints.	Tabular summary of boundaries, deductions, and constraints; "Save and Continue" button.	POST /api/projects/:id/finalize-inputs	On continue: locks feasibility inputs, navigates user to /site-constraints.

Deliverable 6: 30-Day Learning and Rebuild Plan

A structured 4-week engineering schedule designed to take a developer from project scaffolding to a fully defended production deployment:

- Days 1–2 (Stages 0 & 1): Tooling, linting, Vite scaffolding, CSS Modules.
- Days 3–4 (Stages 2 & 3): Login component, validation, JWT handling, Protected Routes.
- Days 5–7 (Stage 4): Projects dashboard, search filtering, "Load More" pagination.
- Days 8–10 (Stages 5 & 6): Mapbox GL JS container, useRef patterns, Geocoder, GeoJSON validation.
- Days 11–12 (Stage 7): Mapbox Draw integration, draw event listeners, HUD metrics panel.
- Days 13–14 (Stage 8): Tree point markers, deduction polygons, containment checks.
- Days 15–17 (Stage 9): Form overlays (GetRegulatoryForm, BusinessConstraints, InputSummary).
- Days 18–19 (Stage 10): Floating Toolbar component, toolActivationRules finite state machine.
- Days 20–21 (Stage 11): useDebounceMapSave hook, timer handling, AbortController cancellation.
- Days 22–24 (Stages 12 & 13): Backend REST integration on port 5001, payload formatting, Toastify.
- Days 25–26 (Stage 14): Unit tests (Vitest), component tests (RTL), and E2E workflow test (Playwright).
- Days 27–28 (Stage 15): Performance profiling, WebGL cleanup verification, Docker containerization.
- Days 29–30 (Review): Practice defending all 100 interview questions.

Deliverable 7: "I Built This Project" Interview Master Script

1. Executive Summary and Problem Statement

"During my software engineering internship at Archonet, I worked on the frontend for Planwise, a specialized site feasibility and generative design platform for the real estate sector. In

traditional real estate acquisition, evaluating whether a land parcel is financially and physically viable takes four to six weeks of manual effort between survey teams, architects, and financial analysts. Planwise compresses that analysis from weeks down to minutes. My core responsibility was building the interactive frontend: the user authentication interface, the project management dashboard, the dynamic constraint forms, and the core spatial workspace—integrating Mapbox GL JS and Mapbox Draw to allow developers to digitize parcel boundaries, place tree preservation markers, and calculate net developable areas in real time."

2. Architecture and Technical Implementation

"Architecturally, the application is a decoupled React single-page application that coordinates a hardware-accelerated WebGL map canvas with standard React component state, communicating with a backend REST API service over HTTP. One of the most critical architectural challenges was managing the lifecycle of the Mapbox canvas alongside React's rendering cycle. Mapbox GL JS runs its own imperative rendering loop directly on the GPU. Storing the map instance in standard component state causes every render to risk re-instantiating the canvas, which exhausts the browser's WebGL contexts and crashes the application with a `CONTEXT_LOST_WEBGL` error. I resolved this by storing the Mapbox instance and its drawing plugins inside React `useRef` hooks. This decoupled canvas operations from React's virtual DOM reconciliation, allowing the map to render smoothly at 60 frames per second while React managed the surrounding form overlays and metrics panels."

3. Dynamic Toolbar and State Machine

"Another key challenge was managing tool dependencies on the spatial canvas. A user should not be allowed to place tree markers or outline non-developable zones until a valid parcel boundary has been drawn and validated. Instead of writing scattered boolean checks across individual buttons, I modeled the toolbar as a centralized Finite State Machine using a `toolActivationRules` object. The system tracks the boundary's presence and validity as derived state. When a valid boundary polygon is completed, the state machine transitions to unlock environmental tools and constraint forms, while keeping disabled tools visually clear and accessible with informative tooltips."

4. Debounced Auto-Save and Concurrency Control

"Because editing parcel boundaries involves dragging vertices across the canvas, Mapbox Draw fires update events on every mouse movement frame—up to 60 times per second. Bombarding the backend with an HTTP request on every event would saturate network connections, exhaust server resources, and introduce out-of-order write race conditions. To solve this, I designed a debounced auto-save engine using a custom hook. The hook holds a timer reference in a `useRef` that resets on every vertex adjustment, dispatching the update call only after a 1000ms pause in user activity. In addition, I linked each request to an `AbortController`. If a user makes another edit while a previous auto-save is still in flight, the pending request is aborted immediately. This guaranteed that earlier, slower requests could never overwrite newer boundary coordinates in the database."

5. Honest Attribution and Engineering Reflection

"To be clear about my contributions: I personally built the client-side architecture—the Login authentication flow with JWT handling, the paginated Projects dashboard, the NewProject modal, the MapPage drawing and HUD metrics systems, the dynamic Toolbar state machine, the constraint forms, and the debounced auto-save hook. I worked closely with backend engineers to define the API payload schemas, such as the stringified GeoJSON structures. The downstream generative layout algorithms that synthesize modular apartment floor plans and the 3D massing extrusion calculations were developed by Archonet's core algorithmic engineering team; my role was capturing, validating, and persisting the clean spatial and regulatory data that fed those generation engines. If I were rebuilding this system today, I would upgrade from JavaScript to TypeScript for compile-time type safety across spatial schemas, replace localStorage with secure HttpOnly cookies and refresh token rotation, and adopt TanStack Query to manage server state and caching. Building this platform gave me practical experience in integrating complex WebGL rendering engines with modern React architectures."

Works cited

1. About Archonet | Real Estate Feasibility Platform, <https://archonet.in/about> 2. LandWise - Land Feasibility Software - Archonet, <https://archonet.in/landwise> 3. UDCPR Schemes for Real Estate Feasibility (Part 1) - Archonet, <https://archonet.in/blog/understanding-udcpr-city-scheme-part-1> 4. FSI Efficiency: Plot Shape, Setbacks & Road Width | Archonet, <https://archonet.in/blog/fsi-efficiency-setbacks-plot-road-width> 5. Archonet: Real Estate Feasibility Software India, <https://archonet.in/> 6. LandWise Pricing and FinWise Plans - Archonet, <https://archonet.in/pricing> 7. FinWise - Real Estate Financial Modelling - Archonet, <https://archonet.in/finwise>